Introduction à la programmation sous Android

Christophe Renaud

M2 Informatique

Année 2024-2025

Version 6.5.4 du 07/01/2025

Objectifs du cours

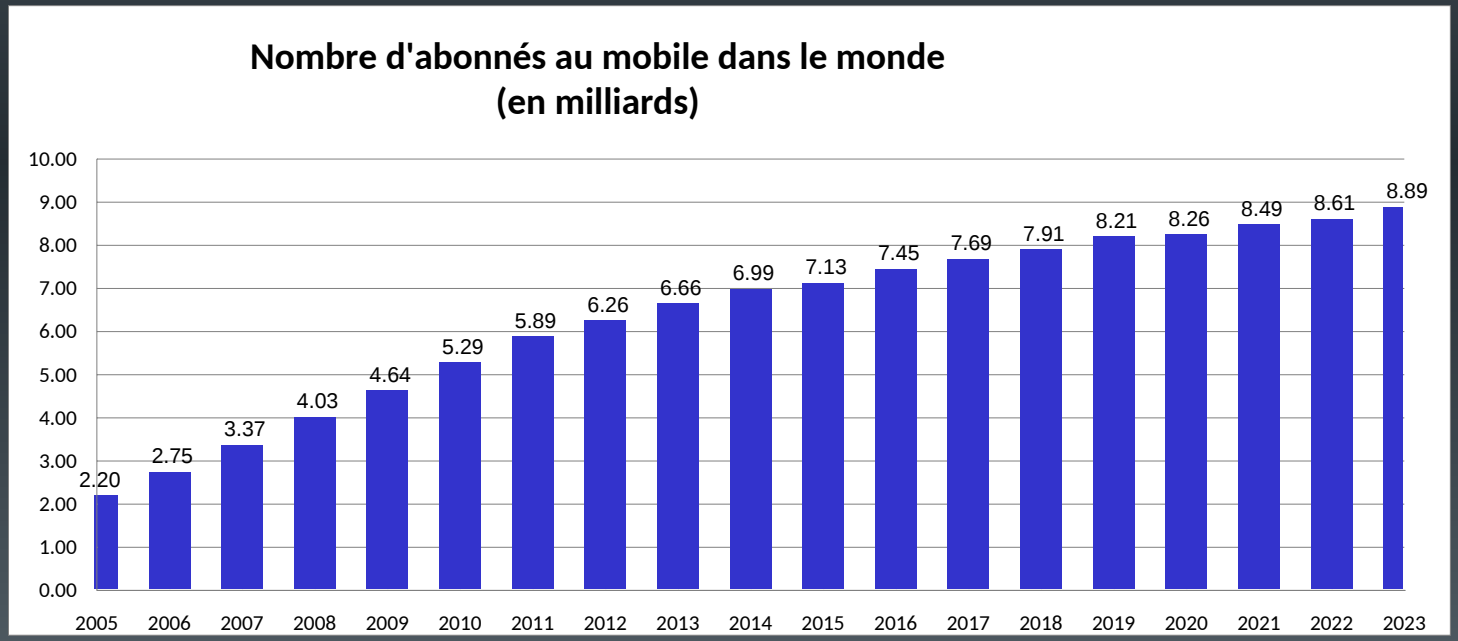
- Connaître les bases de la programmation sous Android
  - Environnement de développement (Android Studio)
  - Architecture d'une application
  - Modèle d'exécution

Plan du cours

- Introduction
- Architecture d'une application Android
- Les activités
- Définir une interface graphique
- Les intentions explicites
- Les intentions implicites
- Les menus
- Les listes
- Les permissions
- Les content providers

Développement mobile (1)

- Pourquoi développer des applications mobiles ?



https://www.itu.int/en/ITU-D/Statistics/Pages/stat/default.aspx

Développement mobile (2)

- Quelles solutions pour le développement ?

Natif

- IOS (objective C / swift)
- Android (java, kotlin)

+ bien adapté à l'OS ciblé
+ ergonomie/design optimal
+ notification push
+ autonomie réseau
- coût de développement (multiplateforme)

Hybride

- React Native (javascript)
- Ionic react (js, html, css)

+ Temps de développement réduit
+ Maintenance facilitée
- Performances moins stables

Web

- Vue.js

+ rien à installer
+ dvt rapide et portable
- fonctionnement sous-optimal
- besoin d'une connexion internet
- pas de notifications push

Flutter (langage Dart)

Android (2)

- Système d'exploitation à destination des dispositifs mobiles
  - Téléphones, tablettes, téléviseurs, montres, voitures, objets connectés
- Caractéristiques :
  - Opensource (licence Apache), gratuit, flexible
  - Basé sur un noyau linux
  - Inclut les applications de base (téléphone, sms, carnet d'adresse, navigateur, etc.)
  - Un ensemble important d'API (OpenGL, media, etc …)
  - Un SDK basé sur un sous-ensemble de JAVA (autres langages disponibles : Kotlin, C, C++, …)
  - Une machine virtuelle (Dalvik) qui exécute la majorité des applications
    - Remplacée par ART depuis la version 5.0 d'Android

Android (3)

- Historique :
  - Créé en 2005 par la société Android
  - Rachat en 2007 par Google
  - 20 versions depuis la 1.0 (Apple Pie) en 2008 jusqu'à la 14.0 en 10/2023.
  - Une version = 1 API
    - Apparition de nouvelles fonctionnalités
    - Modification de fonctionnalités existantes
    - Disparition de certaines fonctionnalités

version	Nom de code	API
4.0	Ice Cream Sandwich	15
4.1	Jelly Bean	16
4.2	Jelly Bean	17
4.3	Jelly Bean	18
4.4	Kitkat	19
5.0	Lollipop	21
5.1	Lollipop	22
6.0	Marshmallow	23
7.0	Nougat	24
7.1	Nougat	25
8.0	Oreo	26
8.1	Oreo	27
9.0	Pie	28
10.0	Q (Quince Tart)	29
11.0	R (Red Velvet Cake)	30
12.0	S (Snow Cone)	31-32
13.0	T (Tiramisu)	33
14.0	U(Upside Down Cake)	34
15.0	V(Vanilla Ice Cream)	35

Android (1)

- Pourquoi développer sous Android ?

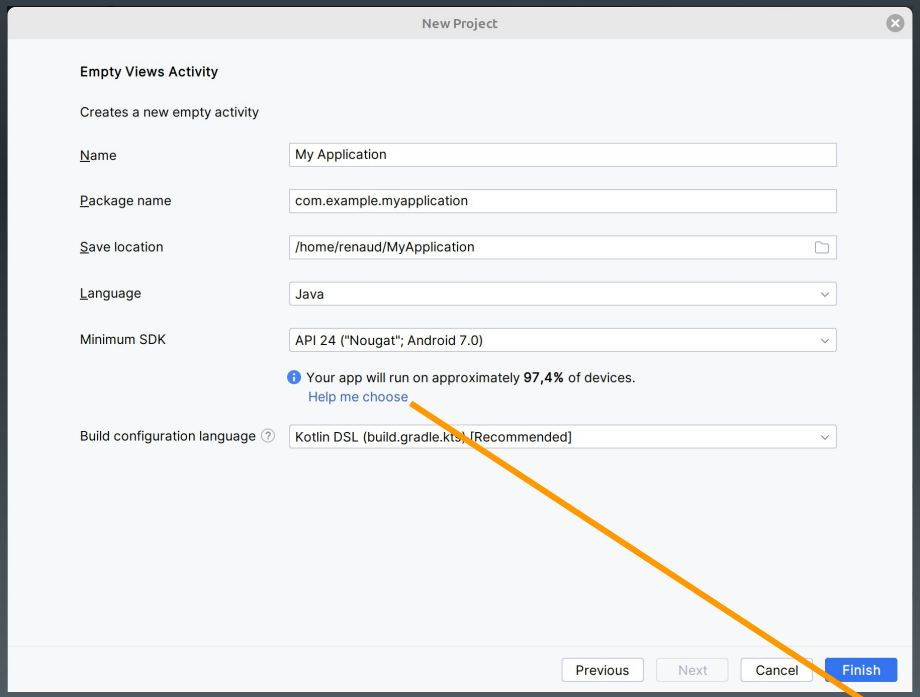
Répartition des ventes de smartphones selon leur OS (en %)

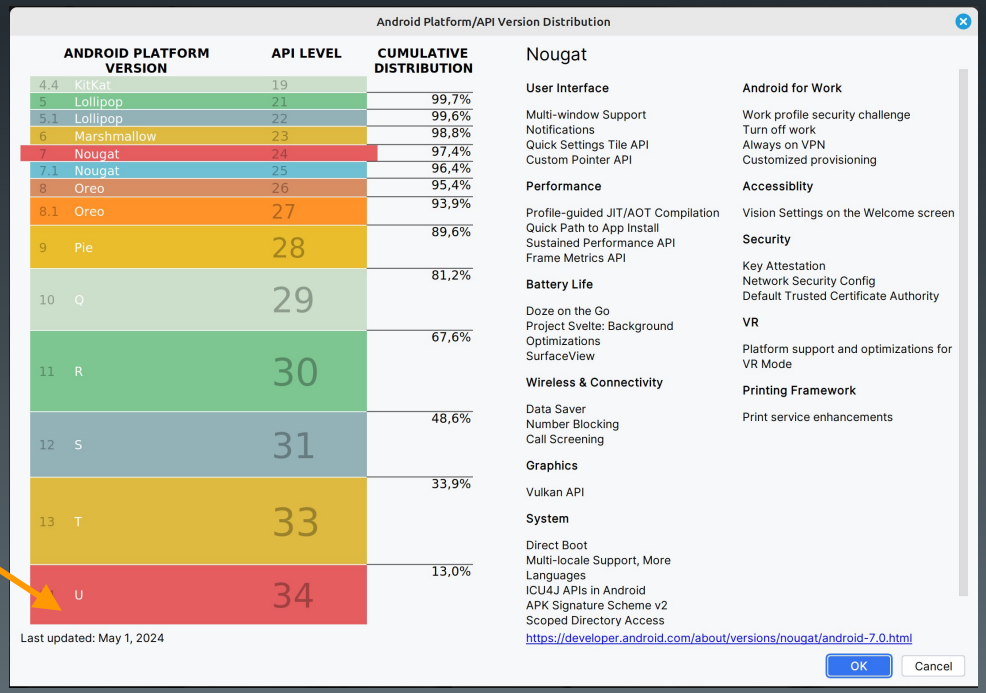
	2019	2020	2021	2022	2023	2024
android	75,47	73,1	71,89	71,48	70,31	71,31
iOS	22,70	26,3	27,34	27,84	28,97	27,98
Autres	1,83	0,6	0,77	0,68	0,72	0,71

Source (octobre 2024) : https://gs.statcounter.com/os-market-share/mobile/worldwide

Android (4)

- Comment assurer la compatibilité ?
  - Vérifier depuis Android Studio lors de la création d'un nouveau projet





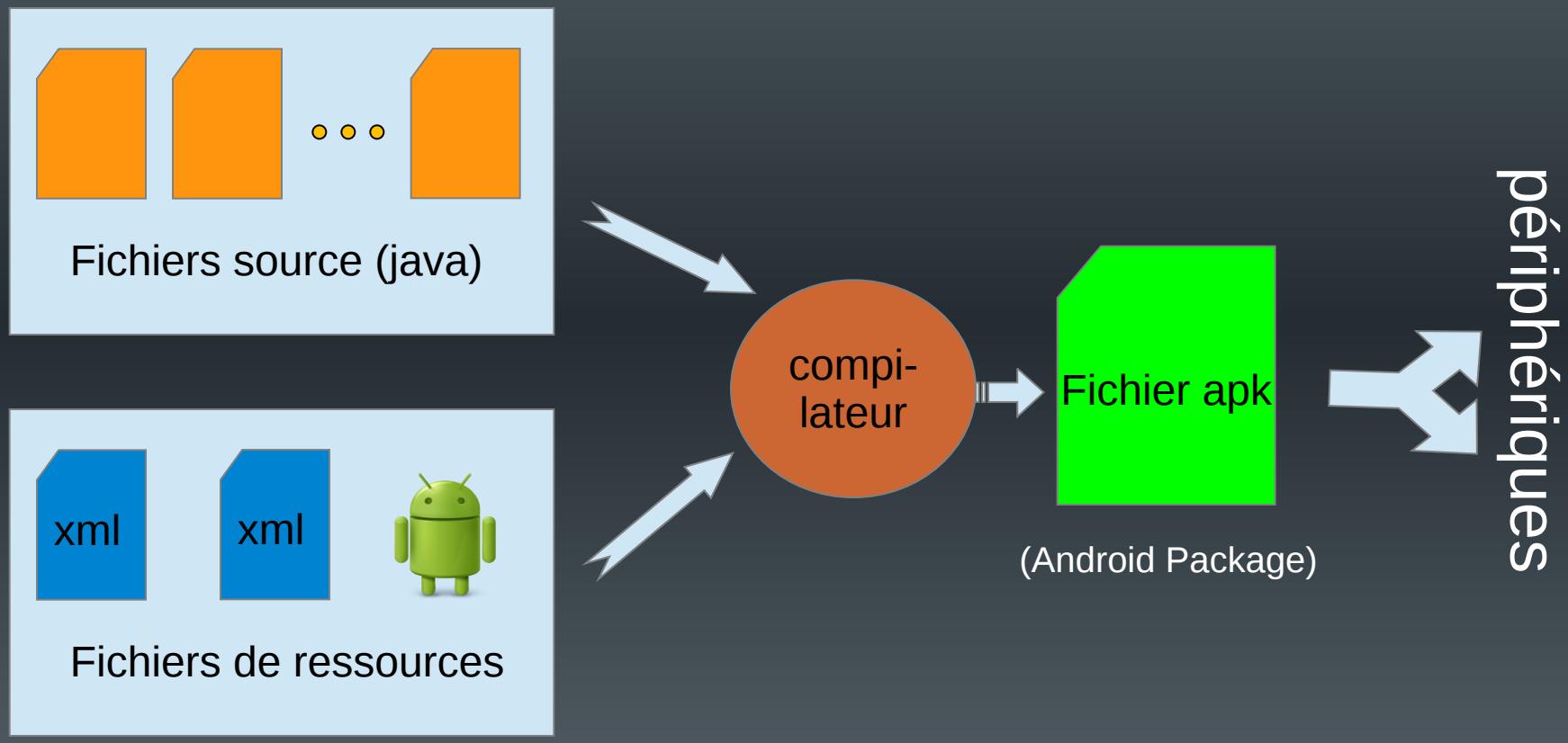
Android (5)

- Les contraintes
  - Hétérogénéité du matériel
    - Processeurs, mémoire
    - Écrans
    - Dispositifs spécialisés
  - Puissance et mémoire limitées
  - Interface tactile
  - Connectivité à internet (disponibilité, rapidité, ...)
  - Développement extérieur au périphérique

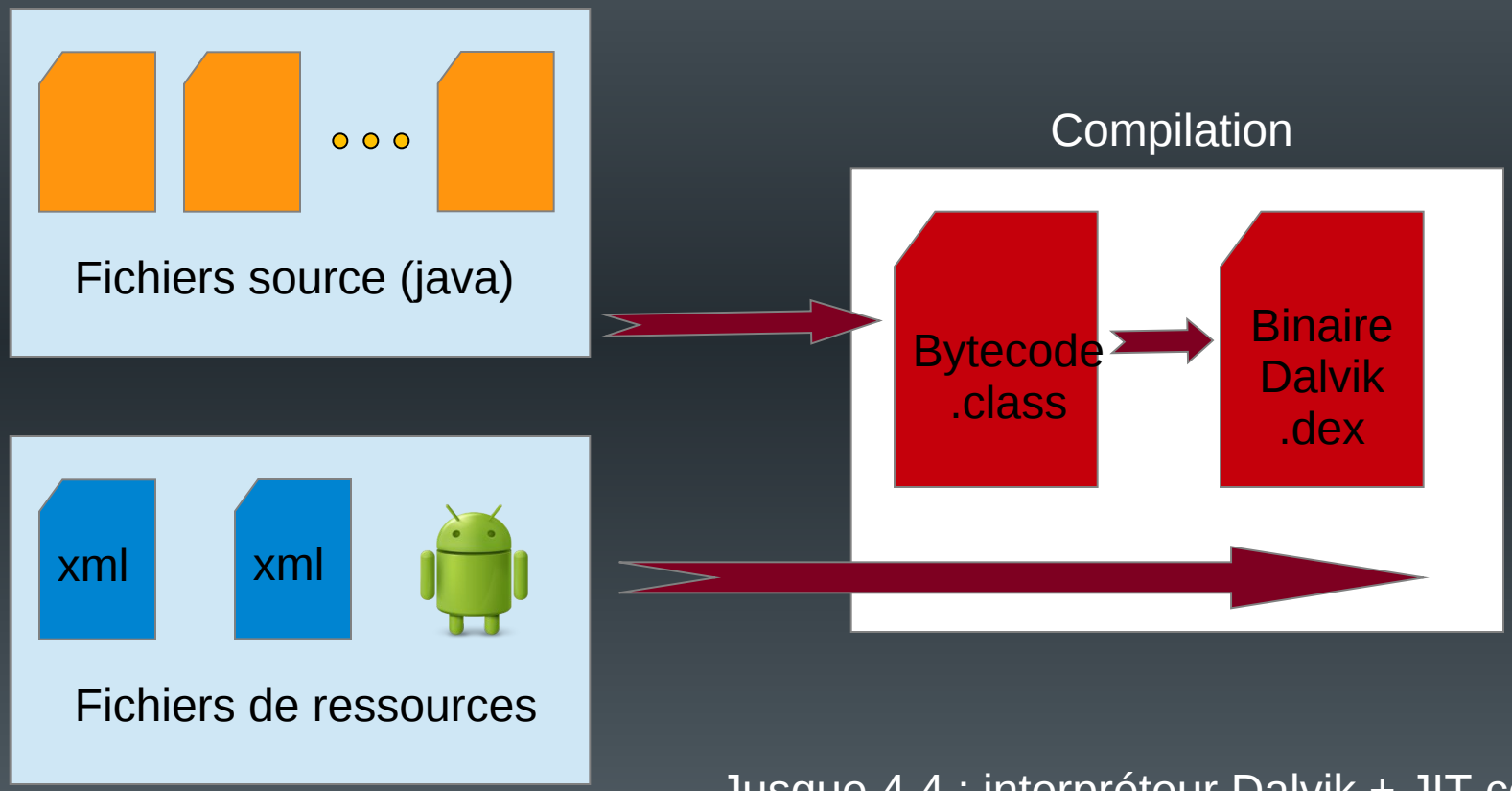
Plan du cours

- Introduction
- Architecture d'une application Android
- Les activités
- Définir une interface graphique
- Les intentions explicites
- Les intention implicites
- Les permissions
- Les menus
- Les listes
- Les content providers

Schéma de développement

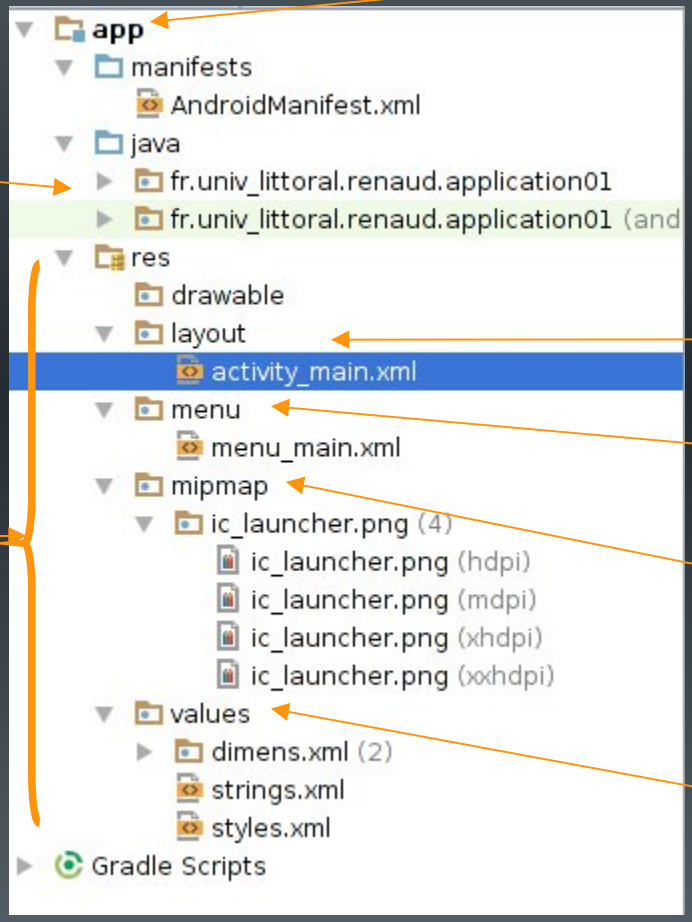


La compilation



Jusque 4.4 : interpréteur Dalvik + JIT compilation de parties « critiques »
À partir de 5.0 : ART (compilation en code natif sur le support)

Architecture d'un projet



Dossier du projet

Sources Java

Mises en page

Menus

Les ressources

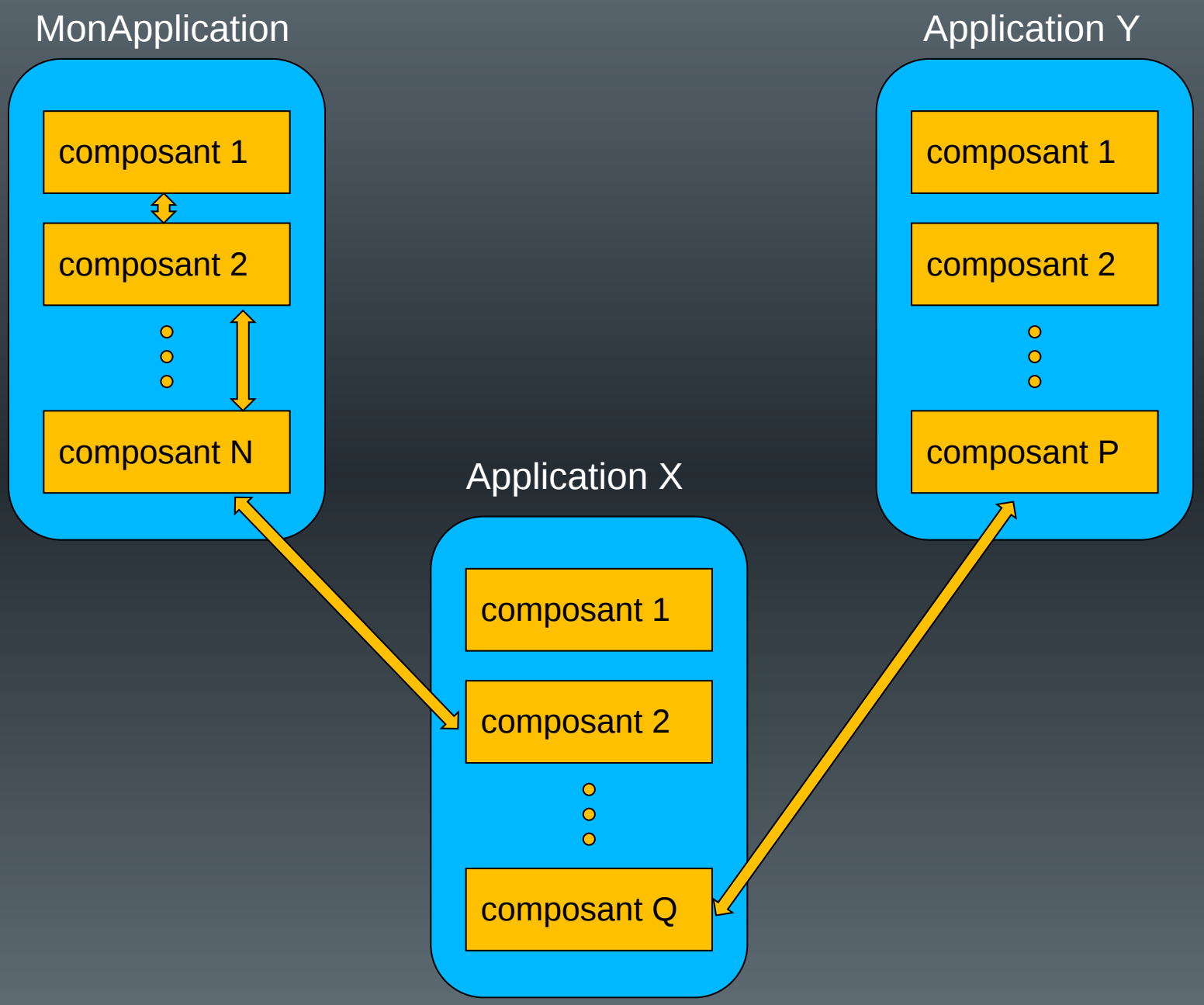
Icônes

Données « constantes »

(sous Android Studio)

Les éléments d'une application

- Une application = {composants}
- Les composants :
  - Existent de manière indépendante
  - Vus comme autant de points d'entrée par le système
    - Pas de « main » dans une application
- Liés au design d'Android :
  - Toute application doit pouvoir démarrer un composant d'une autre application (sous réserve de droits) et récupérer ses « résultats »


MonApplication
composant 1
composant 2
composant N
Application Y
composant 1
composant 2
composant P
Application X
composant 1
composant 2
composant Q

Exemple

- Mon application = application d’effets sur un portrait de l'utilisateur

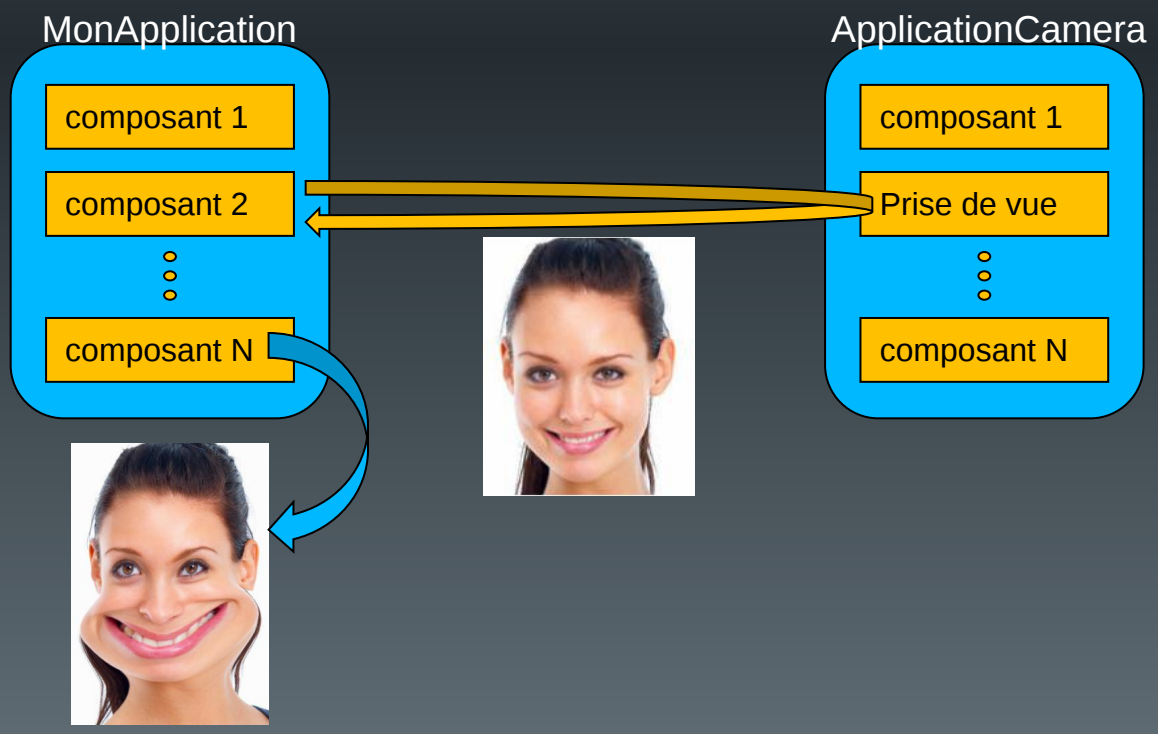
Ma spécialité : traitement d’images

La difficulté : écrire le code de gestion de l'appareil photo embarqué

Source : google play - effets du visage

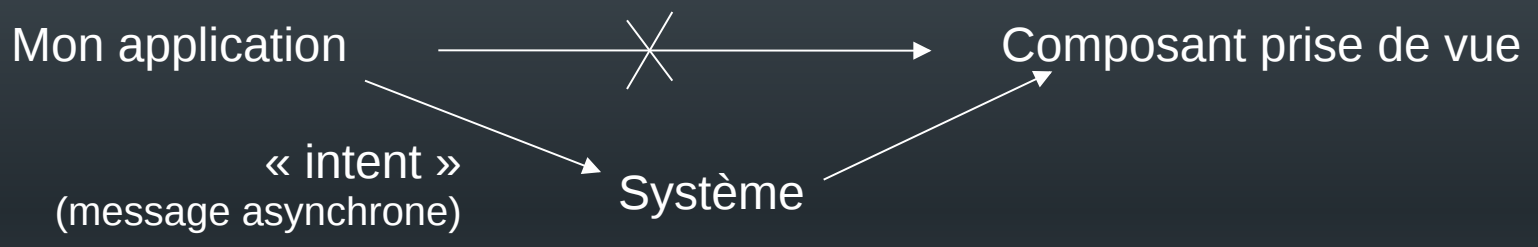
Exemple

- Android :
  - démarrage d'un composant existant permettant la prise de vue
  - Récupération de l'image



Remarques

- Problèmes de droits :



- Problèmes d'information :
  - Le système doit connaître le rôle particulier de certains composants
    - Ce sont les applications qui enregistrent ces informations auprès du système

Les composants

- Les activités (Activity)
  - Un écran avec une interface utilisateur et un contexte
- Les services (Service)
  - Composant sans interface, qui tourne en fond de tâche (lecteur de musique, téléchargement, ...)
- Les fournisseurs de contenu (ContentProvider)
  - I/O sur des données gérées par le système ou par une autre application
- Des récepteurs d'intentions (BroadcastReceiver)
  - Récupération d'informations générales
    - arrivée d'un sms, batterie faible, ...

Les interactions

- Les intentions (*Intent*)
  - Permet d’échanger des informations entre composants
    - Démarrage d’un composant en lui envoyant des données
    - Récupération de résultats depuis un composant
    - Recherche d’un composant en fonction d’un type d’action à réaliser
- Les filtres d'intentions (<intent-filter>)
  - Permet à un composant d'indiquer ce qu'il sait faire
  - Permet au système de sélectionner les composants susceptibles de répondre à une demande de savoir-faire d’une application

AndroidManifest.xml

- Description de l'application
  - Liste des composants
  - Niveau minimum de l'API requise
  - Liste des caractéristiques physiques nécessaires
    - Évite d'installer l'application sur du matériel non compatible (gestion de la visibilité sur Google Play)
  - Liste des permissions dont l'application a besoin
  - Liste des autres API nécessaires
    - ex. Google Map
  - Etc.
- <u>Généré automatiquement par Android Studio</u>

Exemple

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
      package="fr.univ_littoral.renaud.bidon" >

<application
        android:allowBackup="true"
                android:icon="@mipmap/ic_launcher"
                android:label="@string/app_name"
                android:theme="@style/AppTheme" >
                <activity
                android:name=".MainActivity"
                android:label="@string/app_name" >
                <intent-filter>
                        <action android:name="android.intent.action.MAIN" />
                        <category android:name="android.intent.category.LAUNCHER" />
                </intent-filter>
                </activity>
 </application>
</manifest>
```

Les ressources

- Ressources = toutes les données (autres que le code) utilisées par l'application
- Rangées dans le dossier **res**, puis incluses dans l'apk
  - res/drawable et res/mipmap (images en différentes résolutions)
  - Layout (description en XML des interfaces)
  - Menus (description en XML des menus)
  - Values (définitions en XML des constantes utilisées par l'application : chaînes, tableaux, valeurs numériques, etc.)

strings.xml

- Fichier ressources, contenant toutes les chaînes constantes
  - Principalement utilisées pour l'interface

```
<resources>
    <string name="app_name">MyApplication</string>

    <string name="hello_world">Hello world!</string>
    <string name="action_settings">Settings</string>
</resources>
```

Type de la constante

Nom de la constante (permet l'appel depuis l'application ou un autre fichier XML)

Valeur de la constante

Internationalisation

- Objectif :
  - Disposer de plusieurs versions des textes, libellés, etc utilisés par l'application
  - Choix automatique des textes en fonction de la configuration du périphérique
- Principe
  - Dupliquer le fichier strings.xml : 1 version par langue supportée
  - Stocker chaque version dans un dossier spécifique
    - values-xx (ex. values-en, values-fr, …)
  - Géré via Android Studio

```
app/
  res/
    values/
        strings.xml
    values-en/
        strings.xml
    values-fr/
        strings.xml
```

La classe R

- Classe générée par l'IDE
  - Permet l'accès aux ressources
  - Créée à partir de l'arborescence présente dans le dossier **res**
  - Elle contient des classes internes dont les noms correspondent aux différents types de ressources (drawable, layout, …)
  - Elle contient des propriétés permettant de représenter l'ensemble des ressources de l'application
- Utilisation en Java :
  - R. type.identificateur

```
<resources>
   <string name="app_name">MyApplication</string>

   <string name="hello_world">Hello world!</string>
   <string name="action_settings">Settings</string>
</resources>
```

R.string.app_name

R.string.hello_world

R.string.action_settings

Référencement des ressources en XML

- Forme générale : @type/identificateur

```
<resources>
   <string name="app_name">MyApplication</string>

   <string name="hello_world">Hello world!</string>
   <string name="action_settings">Settings</string>
</resources>
```

@string/app_name

@string/hello_world

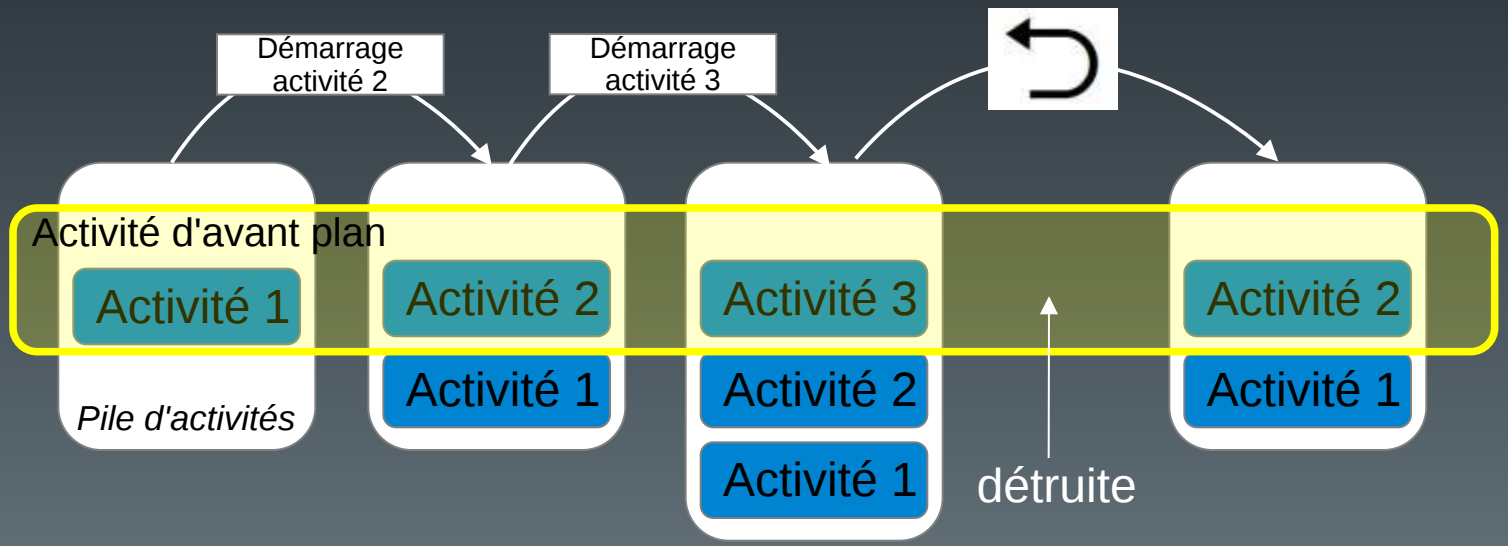
@string/action_settings

Plan du cours

- Introduction
- Architecture d'une application Android
- Les activités
- Définir une interface graphique
- Les intentions explicites
- Les intentions implicites
- Les menus
- Les listes
- Les permissions
- Les content providers

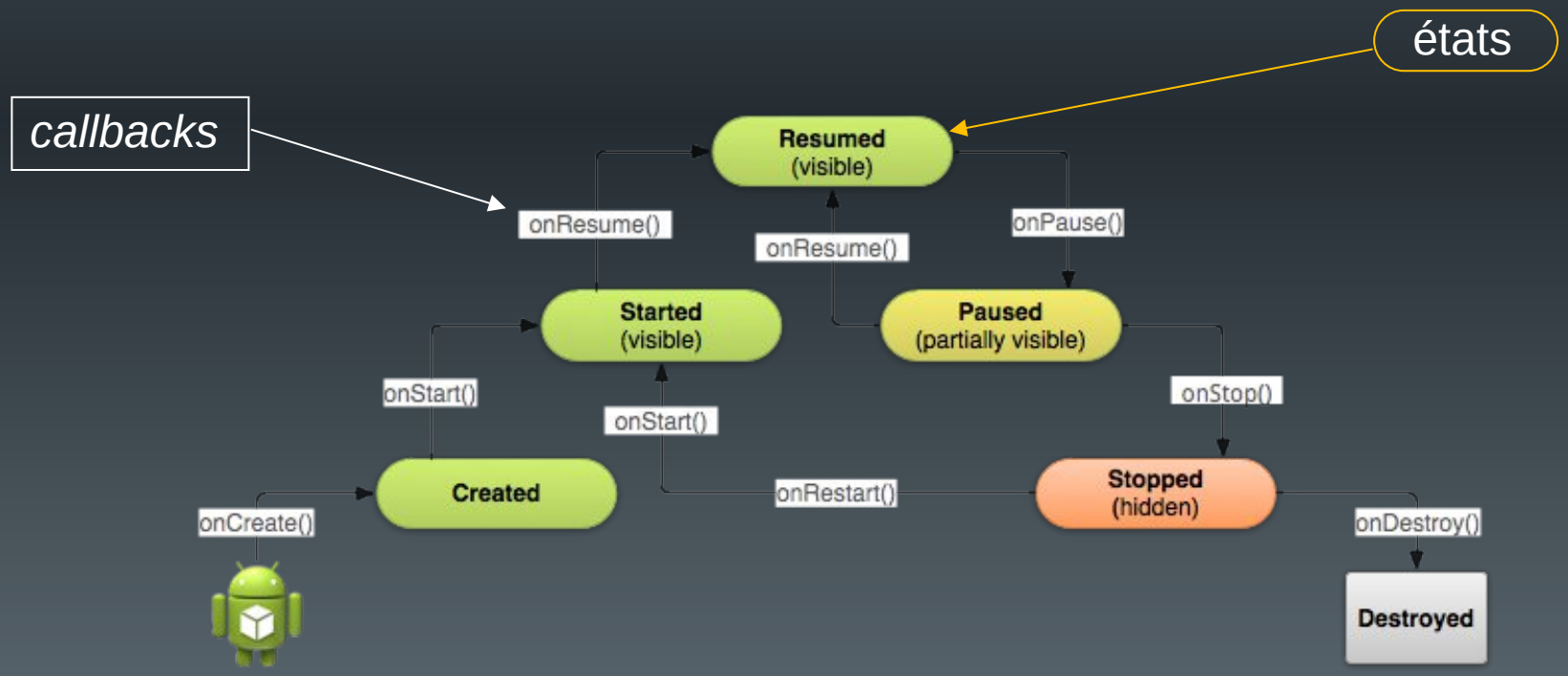
Les activités (1)

- Un composant d'une application, doté d'une interface graphique (IHM) et d'un contexte
- Une activité à la fois visible de l'utilisateur
  - Pour une même application
  - Pour des applications différentes
- Empilement des activités



Les activités (2)

- Cycle de vie
  - Une activité peut se trouver dans différents états en fonction des actions du système et/ou de l'utilisateur :



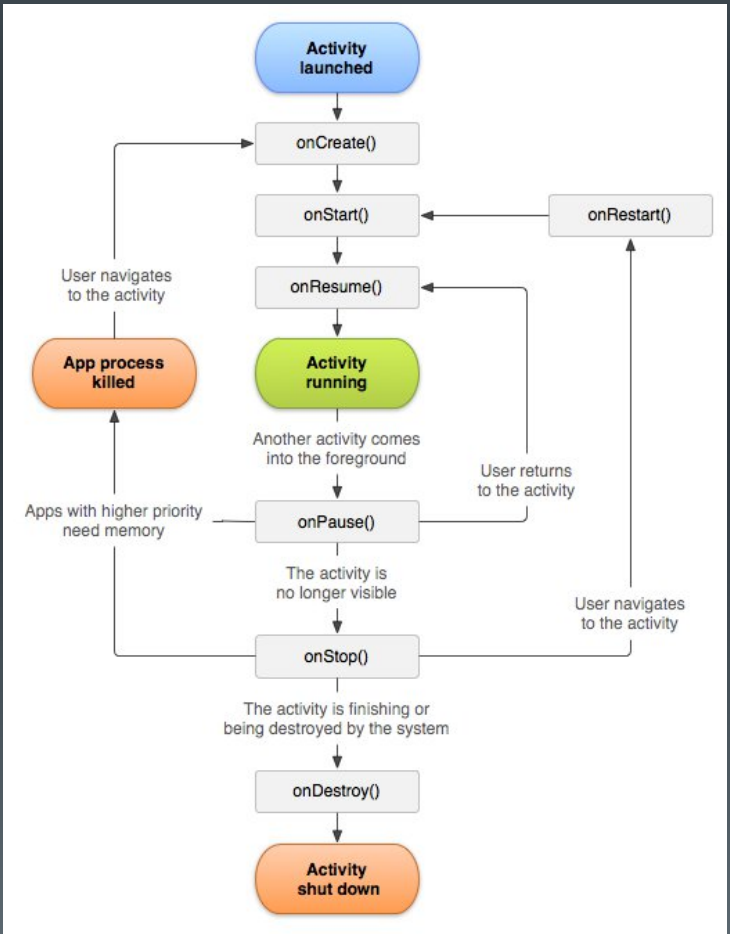
Les activités (3)

- Développement
  - Une classe java par activité ;
  - Les ressources associées (layout, menu, etc.) ;
  - La classe hérite de la classe AppCompatActivity ;
  - Génération d'un code minimum par défaut sous Android Studio.

```
…

public class Bidon extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.Bidon);

            // initialisation des variables internes
    …
    }

    …
}
```

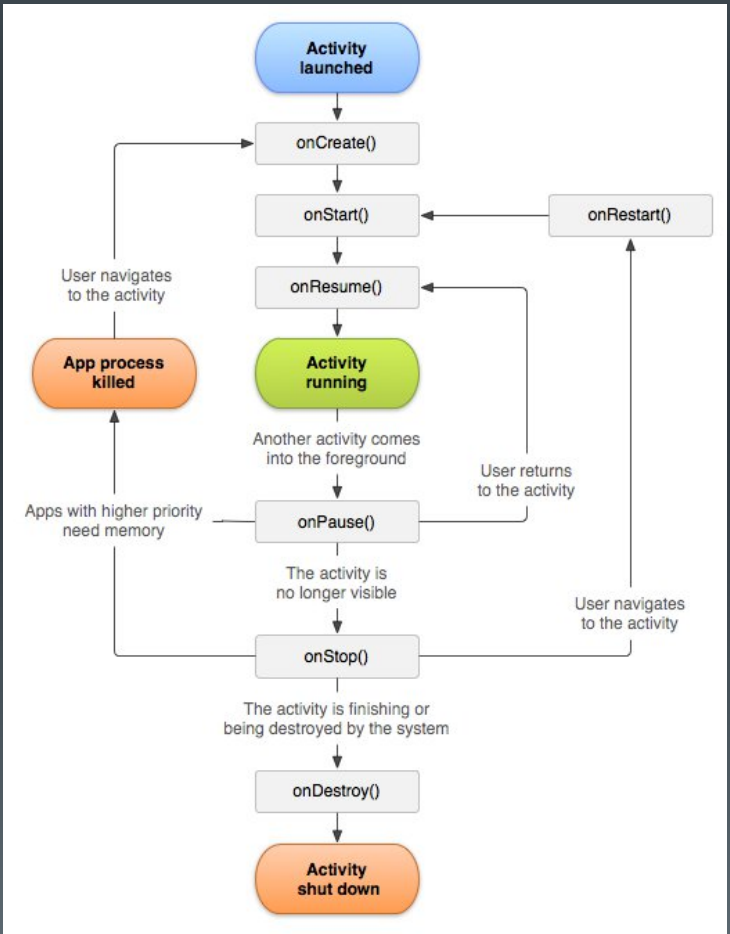
Les activités (4)

- D'autres méthodes peuvent être surchargées, en précisant ce qui doit être fait quand :
  - protected void onDestroy()
    - L'activité se termine
    - Libérer toutes les ressources utilisées
  - protected void onStop()
    - L'activité n'est plus visible
    - Stopper les ressources qui ne sont plus visibles (ex. animations)
  - protected void onPause()
    - L'activité n'est plus au premier plan mais est encore visible (superposition d'une fenêtre de dialogue, multifenêtrage)
    - Stopper les ressources non utilisées (ex. capteur GPS, camera, …)



Les activités (4)

- D'autres méthodes peuvent être surchargées, en précisant ce qui doit être fait quand :
  - protected void onStart()
    - l'activité devient visible
    - Redémarrer les ressources stoppées dans onStop()
  - protected void onResume()
    - l'activité est au premier plan
    - Redémarrer les ressources stoppées dans onPause()
  - protected void onRestart()
    - l'activité redevient visible



Les activités (5)

- Destruction de l'application par le système
  - Cas normal : l'activité est terminée. Le système récupère les ressources, en particulier la mémoire
  - Cas spéciaux : suppression d'une activité non active pour des raisons :
    - de limites des ressources ;
    - de changement d'orientation de l'écran ou de multifenêtrage
  - Le système doit sauvegarder l'état de l'activité, pour pouvoir la redémarrer dans son état courant
    - Sauvegarde dans un objet **Bundle** : couples (nom_donnée, valeur)
      - Contient les données utilisées par l'interface par défaut
      - Systématique dès que l'activité n'est plus visible
    - Surcharge des méthodes de sauvegarde et restauration si d'autres données doivent être sauvées

Les activités (6)

outState

("playerName" , "toto")
("playerPower" , 123.890)
("autreClé01" , autreValeur01)
("autreClé02" , autreValeur02)
…

- Sauvegardes
  - **void onSaveInstanceState(Bundle outState)**

```
public final static String PLAYER_NAME_KEY = "playerName"
public final static String PLAYER_POWER_KEY = "playerPower"
```

Constantes définissant le nom des clés

```
String nomJoueur;
float  puissance;
```

Variables internes

```
// Fonction callback appelée en cas de destruction temporaire de l'activité
@Override
public void onSaveInstanceState(Bundle outState) {

    outState.putString(PLAYER_NAME_KEY, nomJoueur);
    outState.putFloat(PLAYER_POWER_KEY, puissance);

    // appel à la super classe pour sauvegarder les données de l'interface
    super.onSaveInstanceState(outState);
}
```

Sauvegarde de la valeur des variables internes dans le bundle

Les activités (7)

- Récupération
  - **void onCreate(Bundle savedInstanceState)**

```
@Override
public void onCreate(Bundle savedInstanceState) {
   // appel à la super classe pour mettre à jour les données de l'interface
   super.onCreate(savedInstanceState);

   // récupération des données internes
   if (savedInstanceState != null) {
      nomJoueur = savedInstanceState.getString(PLAYER_NAME_KEY);
      puissance = savedInstanceState.getFloat(PLAYER_POWER_KEY);
}
```

  - **void onRestoreInstanceState (Bundle savedInstanceState)**

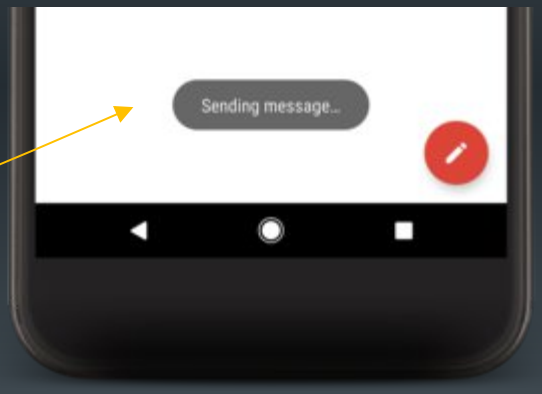
```
// Callback appelé après onStart() uniquement si un appel à onSaveInstanceState()
//  a été fait. Le bundle savedInstanceState est le même que pour onCreate()
@Override
public void onRestoreInstanceState(Bundle savedInstanceState) {
      nomJoueur = savedInstanceState.getString(PLAYER_NAME_KEY);
      puissance = savedInstanceState.getFloat(PLAYER_POWER_KEY);
}
```

TP 01

Remarques

https://www-lisic.univ-littoral.fr/~renaud/

- Affichage de messages de mises au point
  - Possibilité d'utiliser *System.out.println*
    - Affichage dans la console d'Android Studio
  - La classe Toast



Source : develop.android.com

Le message

Toast.*makeText*(**this**, **"Sending message…"**, Toast.***LENGTH_SHORT***).show();

Le contexte

La durée d'affichage

Le déclenchement de l'affichage

Plan du cours

- Introduction
- Architecture d'une application Android
- Les activités
- Définir une interface graphique
- Les intentions explicites
- Les intentions implicites
- Les menus
- Les listes
- Les permissions
- Les content providers

Quelques règles de base

- Interface = seul contact de l'utilisateur
  - Faire attirant
  - Faire simple
    - L'application doit être intuitive
    - Éviter les trop longs messages
- Faire ergonomique
  - L'enchaînement des activités doit être rapide
  - L'utilisateur doit toujours connaître l'état courant de l'activité
- Conseils et « matériels » :
  - http://developer.android.com/design

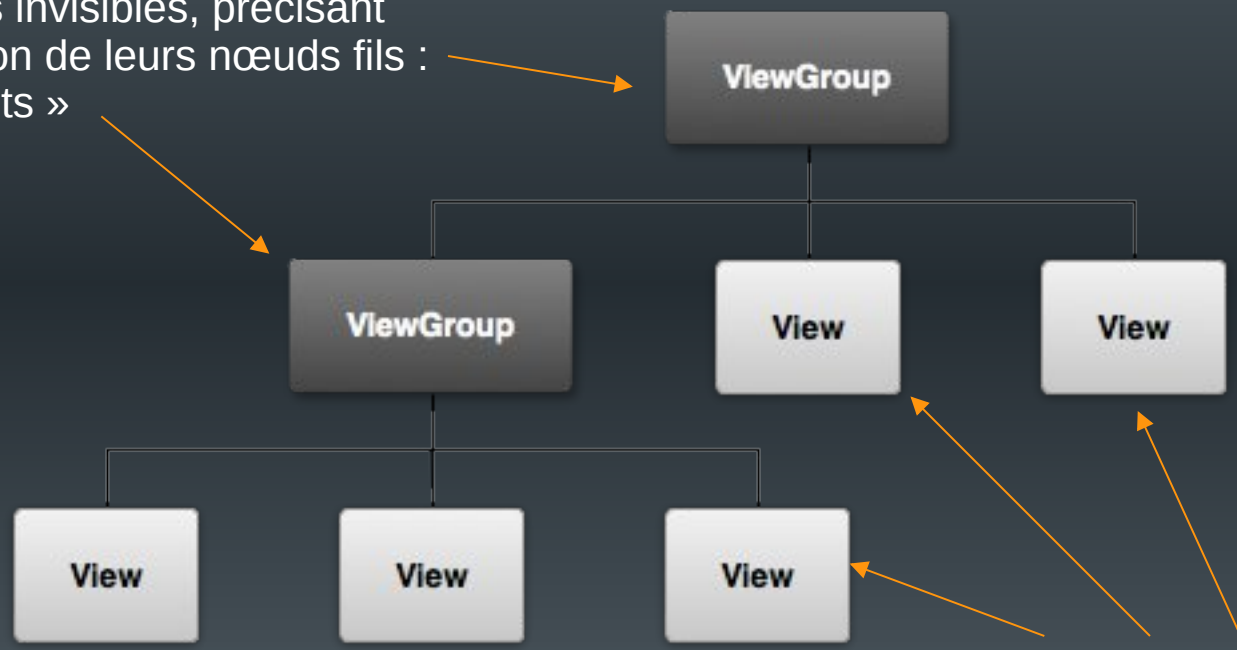
Définir une interface graphique

- Définir les « interacteurs »
  - Objets graphiques visibles par l'utilisateur pour :
    - L'affichage (texte, images, etc.)
    - L'interaction (boutons, cases, champs de saisie, etc.)
- Définir leur mise en page
  - Positions dans l'interface (fixes ou relatives)
- XML ou Java (sauf traitement de l'interaction : Java seul)
    - <u>Privilégier XML</u>
      - Souplesse de mise à jour
      - Permet la prise en compte simplifiée de différents types d'écran

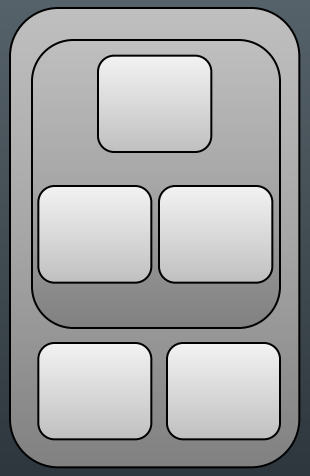
Représentation d'une interface

- Représentation arborescente

Conteneurs invisibles, précisant l'organisation de leurs nœuds fils : les « Layouts »



Source : developer.android.com



exemple

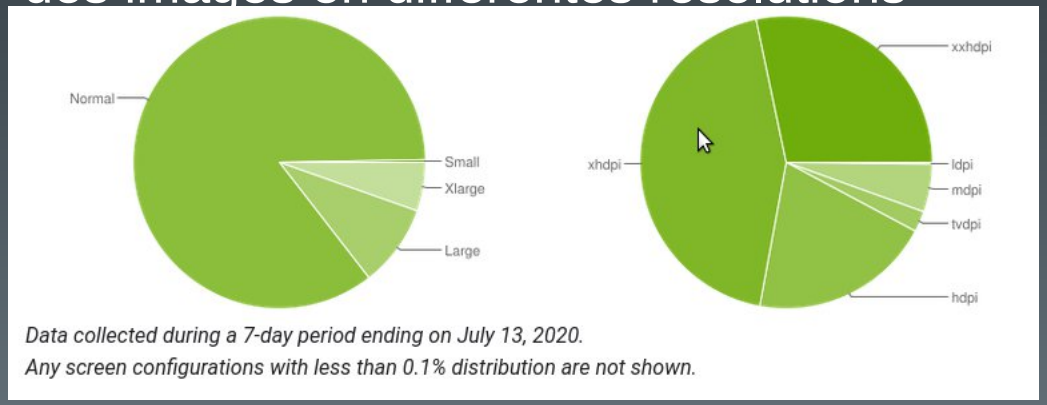
Objets graphiques permettant l'interaction (boutons, zones de texte, etc.) : les Widgets

Les Layouts (1)

- Zone invisible assurant l'organisation automatique des composants graphiques
  - Peuvent être déclarées en XML ou Java
  - Privilégier XML
    - Séparation du code et de la mise en page
    - Souplesse d'adaptation à différents périphériques
- Possèdent des propriétés « intuitives » permettant l'organisation des composants
- Nombreux *layouts* différents
  - Peuvent être imbriqués (cf arborescence)
- Un *layout* doit être chargé dans onCreate()
  - setContentView(R.layout.nom_du_layout)

Les Layouts (2)

- Gestion multi-écrans
  - Différentes tailles
    - small, normal, large, xlarge
  - Différentes densités de pixels
    - ldpi (120dpi), mdpi (160 dpi), hdpi (240 dpi), xhdpi (320 dpi), xxhdpi (480 dpi), xxxhdpi (640 dpi)
  - Prévoir un *layout* par taille (et orientation) de l'écran si nécessaire
    - effets de positionnements relatifs pouvant être gênants
  - Prévoir des images en différentes résolutions



Source
developer.android.com

Les Layouts (3)

- Fonctionnement similaire à l'internationalisation
  - Un sous-dossier spécifique à chaque *layout* et/ou à chaque image

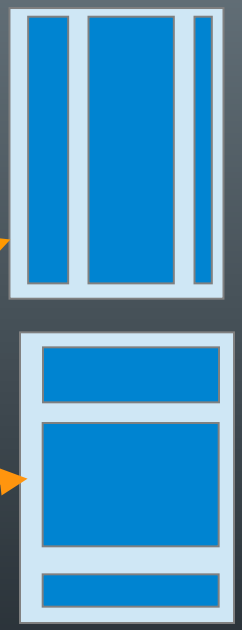
```
MyProject/
    res/
        layout/              # default (portrait)
            main.xml
        layout-land/         # landscape
            main.xml
        layout-large/        # large (portrait)
            main.xml
        layout-large-land/   # large landscape
            main.xml
```

```
MyProject/
    res/
        drawable-xhdpi/
            awesomeimage.png
        drawable-hdpi/
            awesomeimage.png
        drawable-mdpi/
            awesomeimage.png
        drawable-ldpi/
            awesomeimage.png
```

Source : developer.android.com

LinearLayout (1)

- Aligne les nœuds dans une seule direction
  - horizontale (par défaut)
  - verticale



activity_main.xml | MainActivity.java

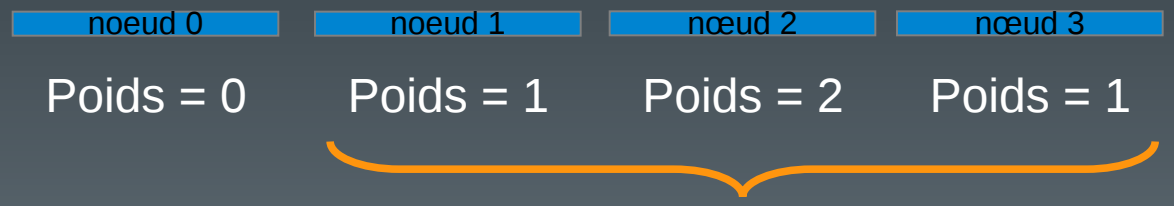
```
1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3      xmlns:app="http://schemas.android.com/apk/res-auto"
4      xmlns:tools="http://schemas.android.com/tools"
5      android:layout_width="match_parent"
6      android:layout_height="match_parent"
7      android:orientation="vertical"
8      tools:context=".MainActivity">
9
10     <TextView
11         android:layout_width="wrap_content"
12         android:layout_height="wrap_content"
13         android:text="Hello World!" />
14
15     <TextView
16         android:id="@+id/textView"
17         android:layout_width="match_parent"
18         android:layout_height="wrap_content"
19         android:text="Coucou" />
20
21 </LinearLayout>
```

Taille du composant :

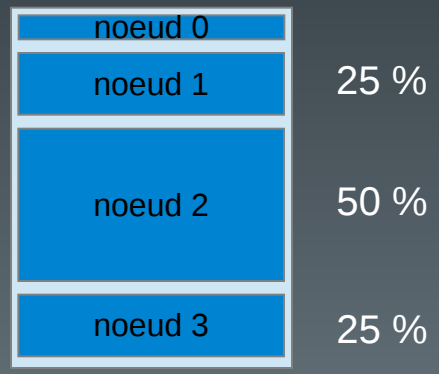
- match_parent : S'adapte à la taille du conteneur parent (ici l'écran)
- wrap_content : S'adapte à la taille de ce qu'il contient (ici une zone de texte)
- *dimension fixe*

LinearLayout (2)

- Modification du « poids » de chaque nœud
  - Permet de changer la taille de la zone occupée par chaque nœud dans l'écran
  - Ajout d'un attribut android:layout_weight à chaque nœud
    - 0 (par défaut) : n'utilise que la zone nécessaire au nœud
    - n>0 : poids du nœud par rapport aux autres nœuds



Répartition au prorata du poids



LinearLayout (3)

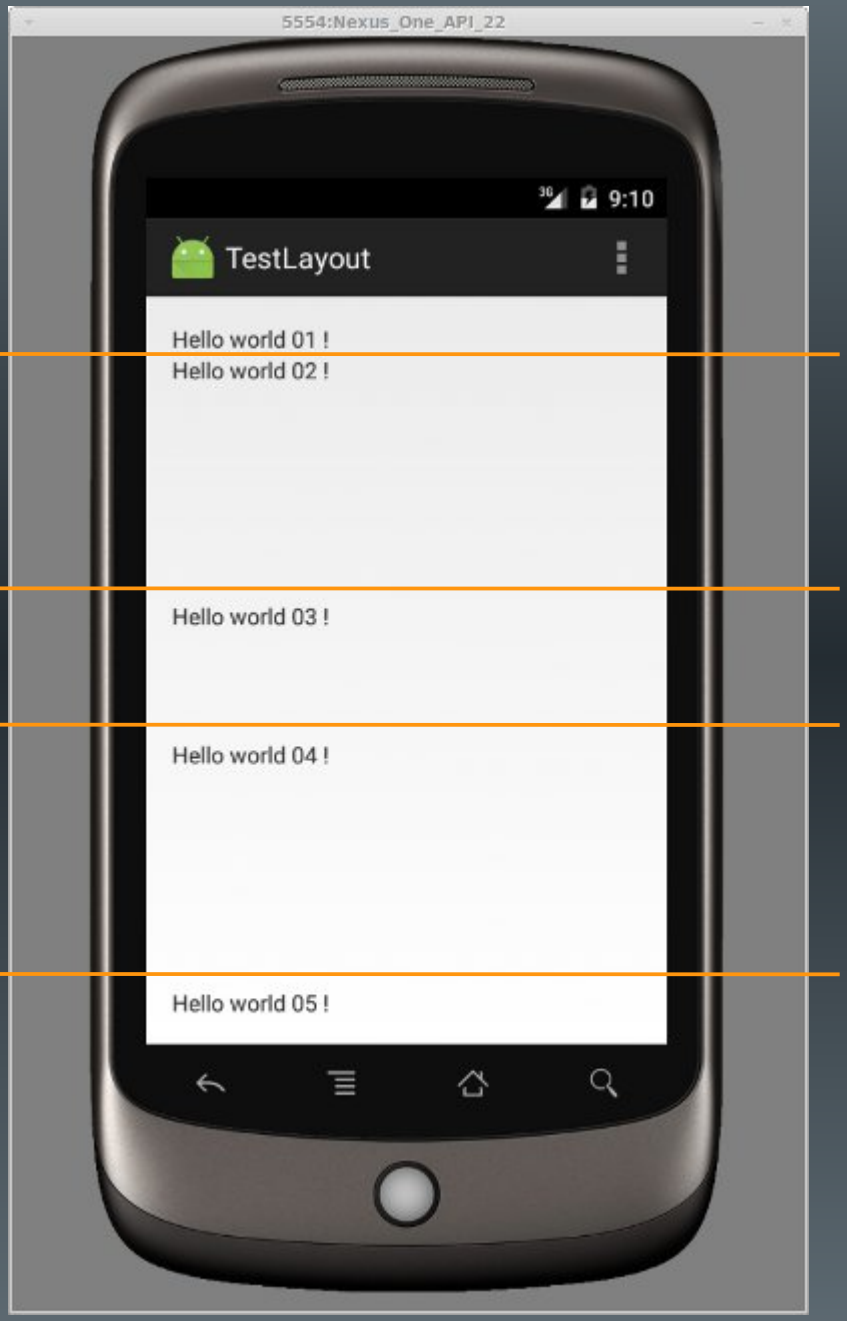
- Exemple

```
<TextView
    android:text="@string/hello_world_01"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    />

<TextView
    android:text="@string/hello_world_02"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_weight="2"
    />
<TextView
    android:text="@string/hello_world_03"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_weight="1"
    />
<TextView
    android:text="@string/hello_world_04"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_weight="2"
    />
<TextView
    android:text="@string/hello_world_05"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    />
```

40 %

20 %

40 %



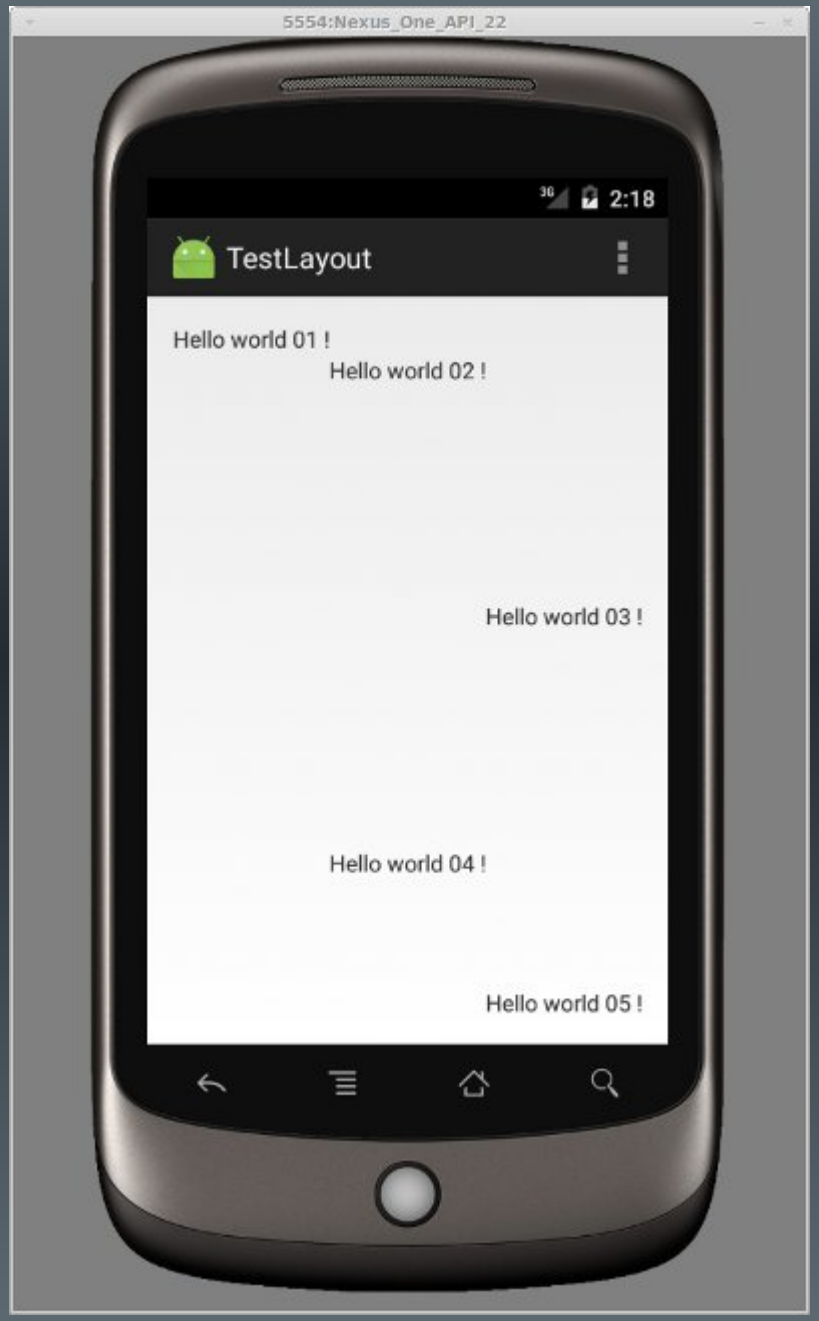
LinearLayout (4)

- Alignement de chaque noeud dans sa zone
  - Ajout d'un attribut android:layout_gravity
  - Nombreuses valeurs possibles :
    - center, center_vertical, center_horizontal
    - left, right, top, bottom
    - Etc. (cf LinearLayout.LayoutParams)

LinearLayout (5)

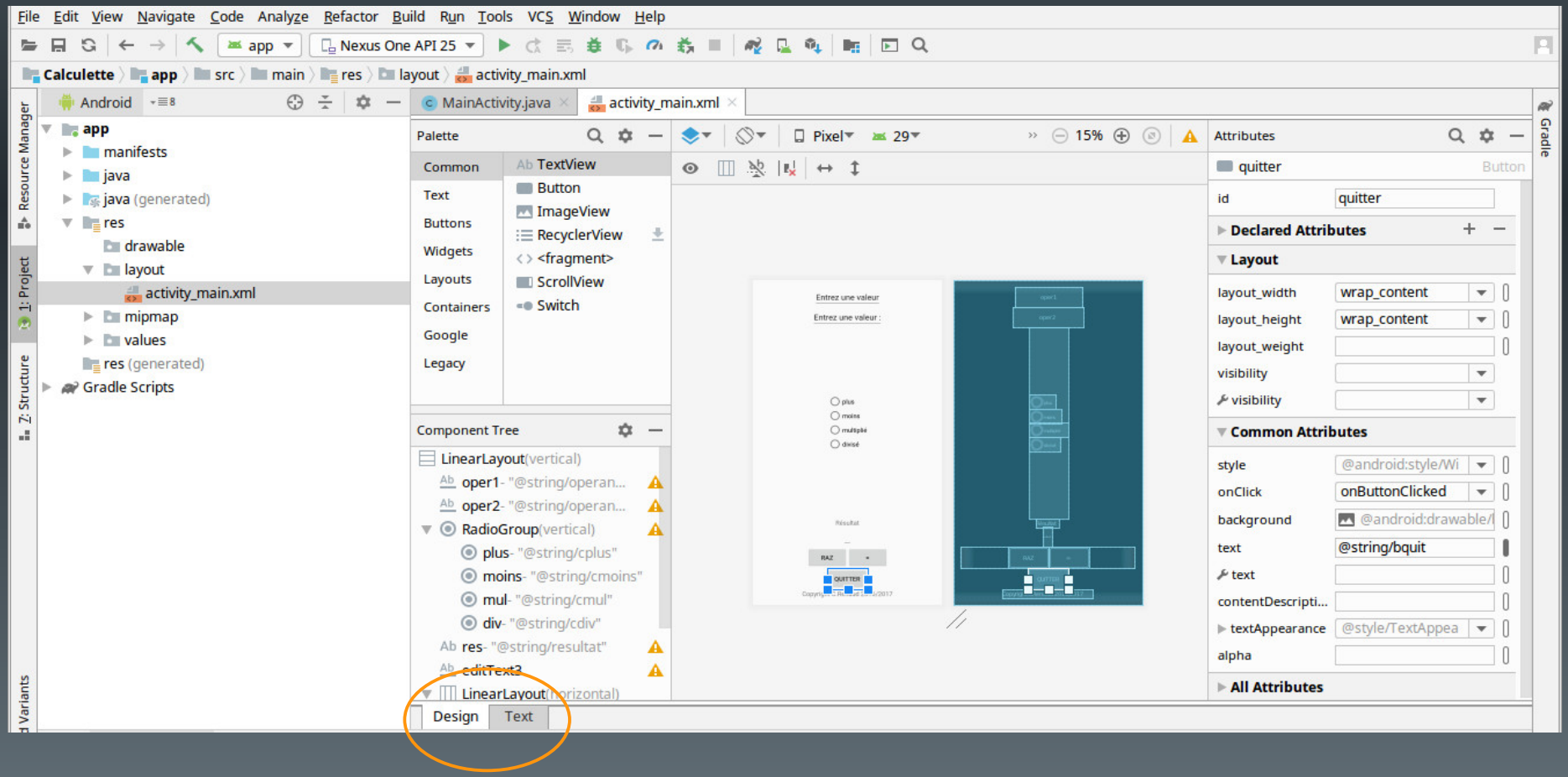
- Exemple

```
<TextView
    android:text="@string/hello_world_01"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    />

<TextView
    android:text="@string/hello_world_02"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="center_horizontal"
    android:layout_weight="2"
    />
<TextView
    android:text="@string/hello_world_03"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="right"
    android:layout_weight="1"
    />
<TextView
    android:text="@string/hello_world_04"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="center"
    android:gravity="center"
    android:layout_weight="2"
    />
<TextView
    android:text="@string/hello_world_05"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="right"
    />
```



Remarque

- Possibilité d'organiser visuellement les layouts sous Android Studio



ConstraintLayout (1)

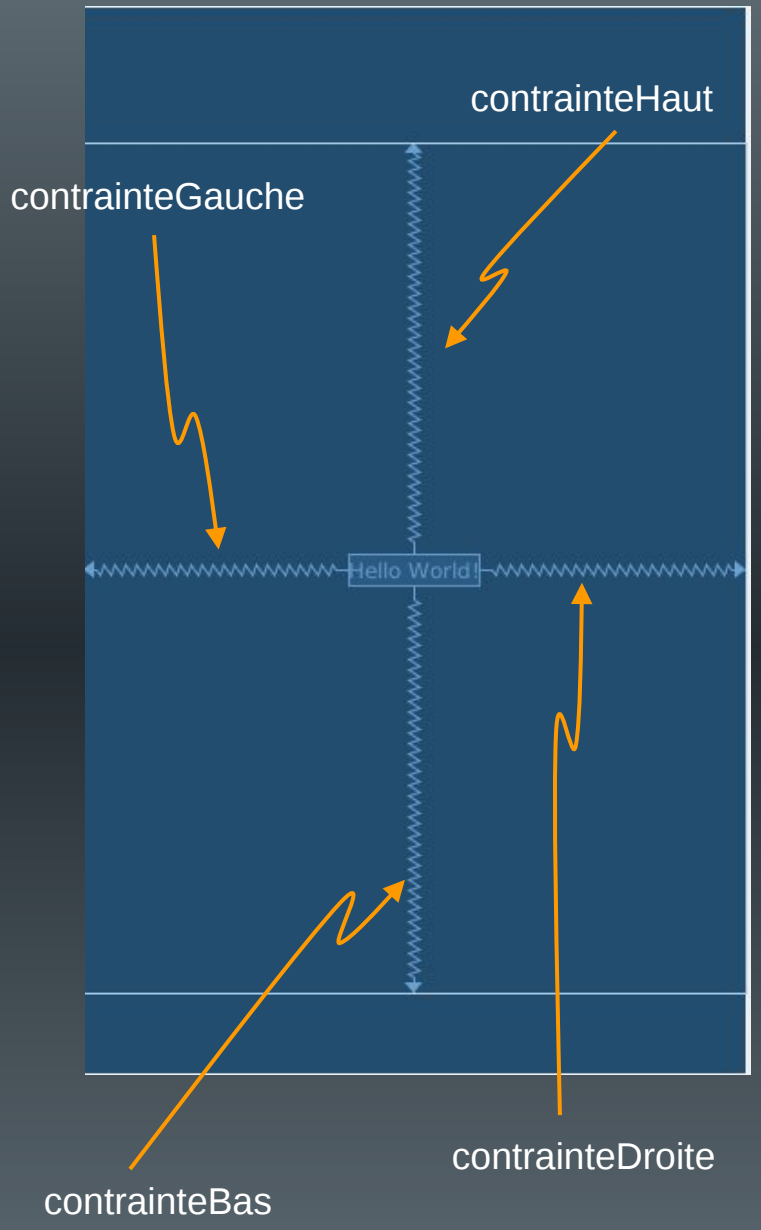
- Objectif :
  - Faciliter la création de *layouts* complexes
  - Réduire l'imbrication de *layouts*
    - Consommatrice de ressources
- Organisation intuitive :
  - Création de relations <u>entre</u> les composants graphiques et <u>avec</u> leur *layout* parent
    - Notion de contraintes
  - Très flexible
  - Bien adapté à l'éditeur graphique

ConstraintLayout (2)

- Contraintes
  - Au moins une contrainte horizontale
  - Au moins une contrainte verticale

Exemple :

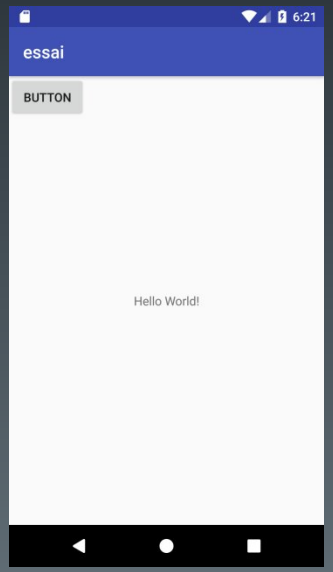
```
<android.support.constraint.ConstraintLayout
xmlns:android=
   "http://schemas.android.com/apk/res/android"
   xmlns:app="http://schemas.android.com/apk/res-auto"
   xmlns:tools="http://schemas.android.com/tools"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   tools:context=".MainActivity">

   <TextView
      android:id="@+id/textView"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:text="Hello World!"
      app:layout_constraintBottom_toBottomOf="parent"
      app:layout_constraintLeft_toLeftOf="parent"
      app:layout_constraintRight_toRightOf="parent"
      app:layout_constraintTop_toTopOf="parent" />

</android.support.constraint.ConstraintLayout>
```

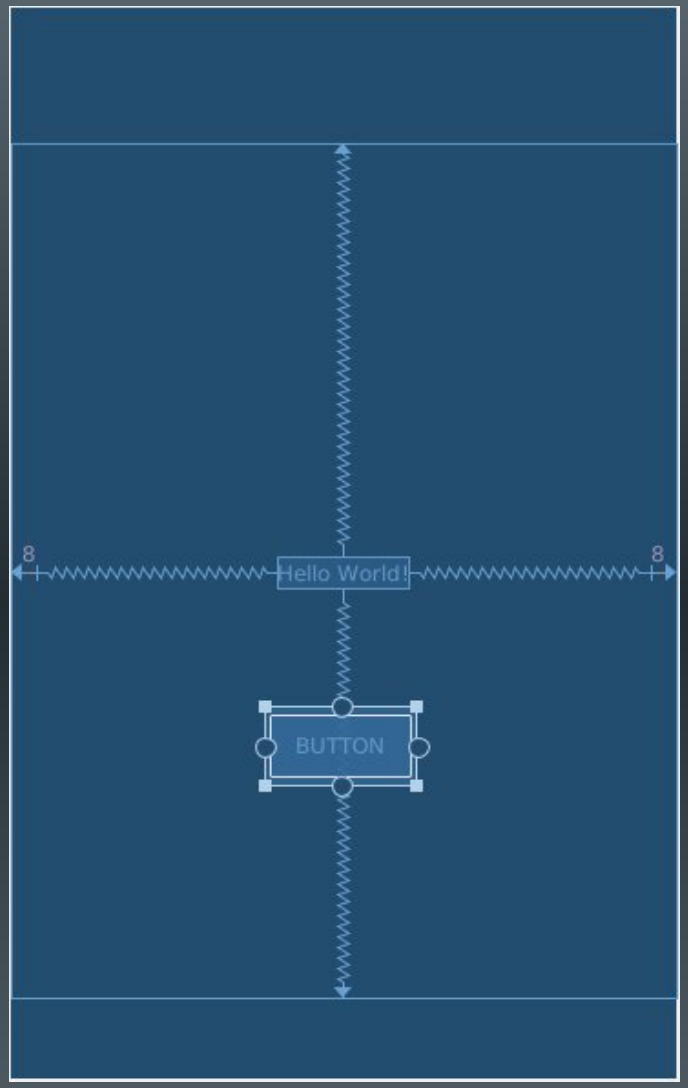


ConstraintLayout (2)

- Remarque
  - Placement <u>sans contrainte</u> d'un composant graphique via l'éditeur graphique
    - Apparaît au bon endroit dans l'éditeur
  - Placé par défaut en (0,0) lors de l'exécution …

```
<Button
    android:id="@+id/button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Button"
    tools:layout_editor_absoluteX="148dp"
    tools:layout_editor_absoluteY="373dp" />
```

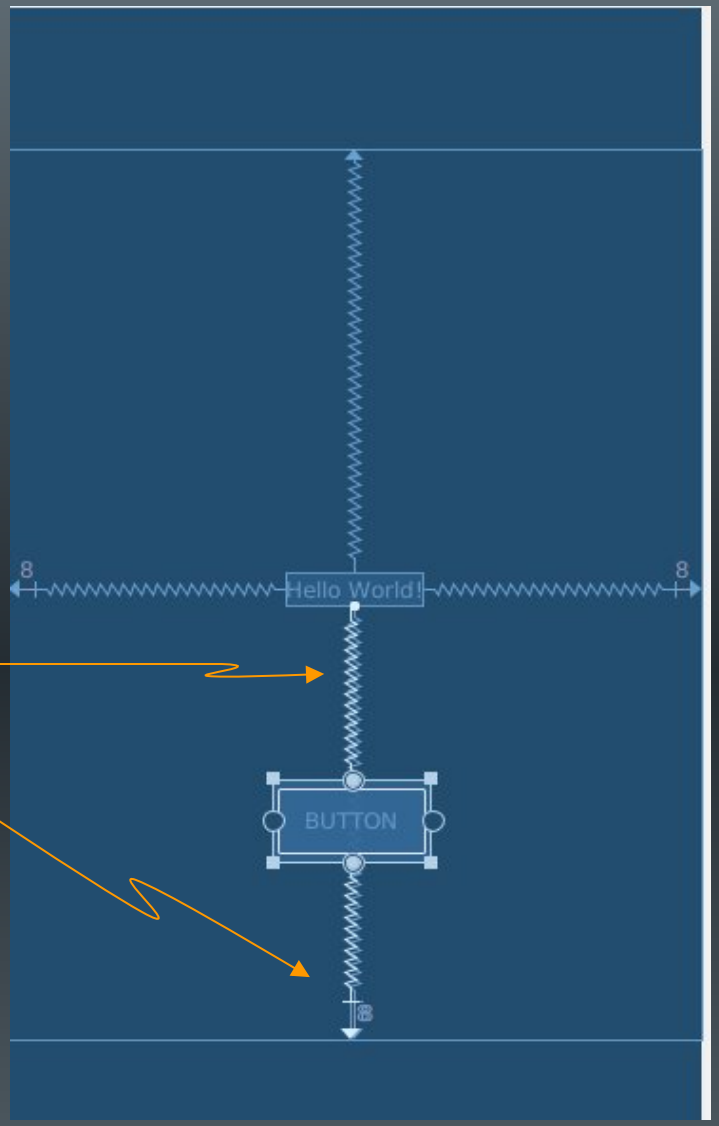


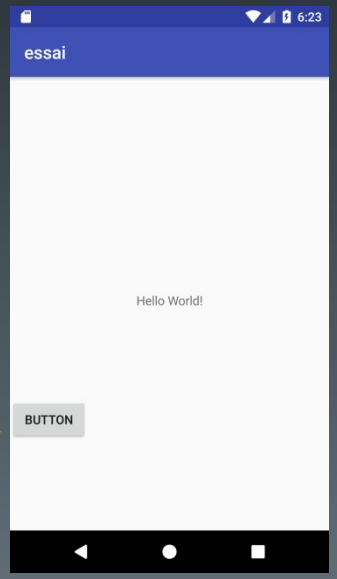


ConstraintLayout (3)

- Ajout d’une contrainte verticale

```
<Button
    android:id="@+id/button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginBottom="8dp"
    android:layout_marginTop="8dp"
    android:text="Button"
    app:layout_constraintTop_toBottomOf="@+id/textView "
    app:layout_constraintBottom_toBottomOf="parent"
   tools:layout_editor_absoluteX="148dp" />
```



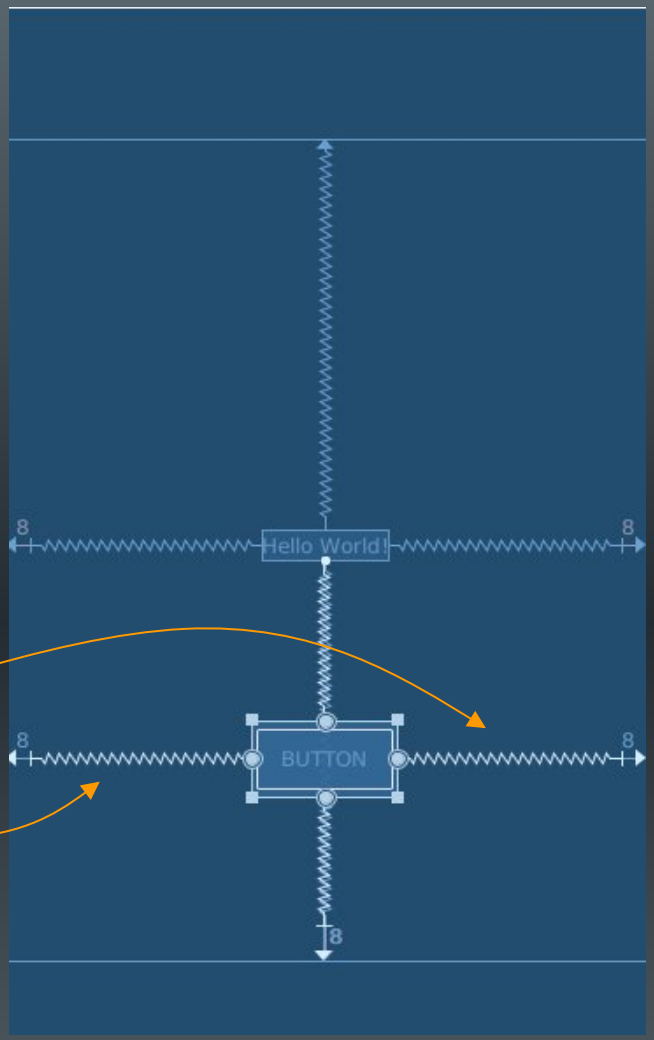


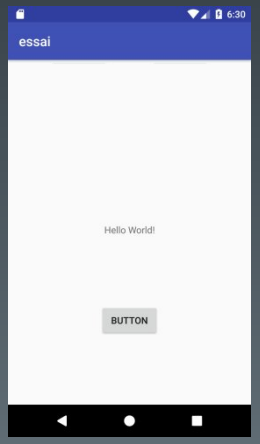
X=0

ConstraintLayout (4)

- Ajout d'une contrainte horizontale

```
<Button
    android:id="@+id/button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginBottom="8dp"
    android:layout_marginEnd="8dp"
    android:layout_marginLeft="8dp"
    android:layout_marginRight="8dp"
    android:layout_marginStart="8dp"
    android:layout_marginTop="8dp"
    android:text="Button"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/textView" />
```



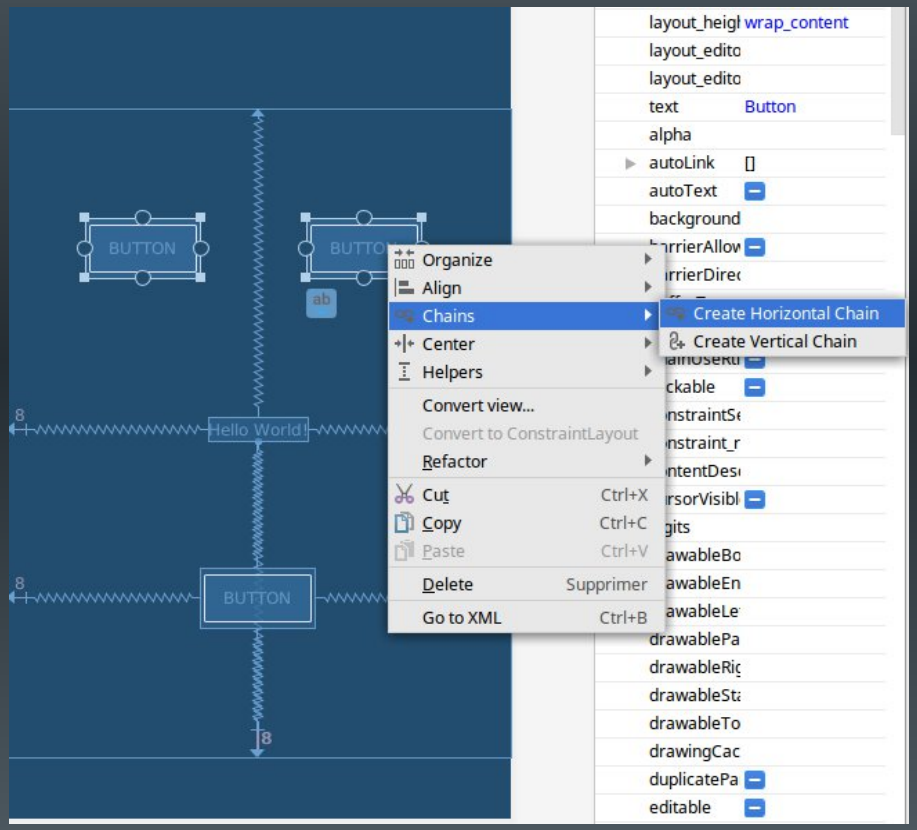


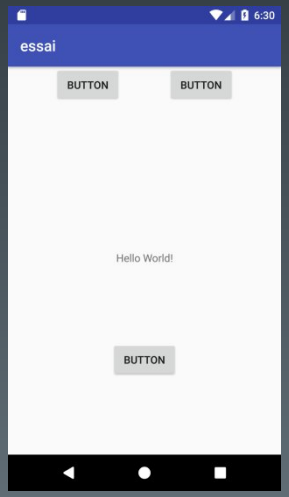
Remarque :

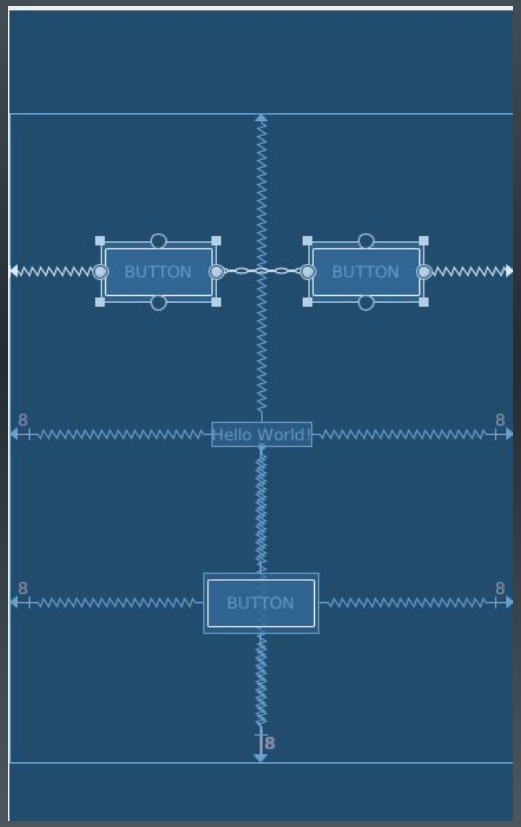
- `@+id/nom` rajoute nom à la classe R uniquement si `nom` n'existe pas
- Utile si le composant nommé est défini après dans le fichier xml

ConstraintLayout (5)

- Création de chaînages

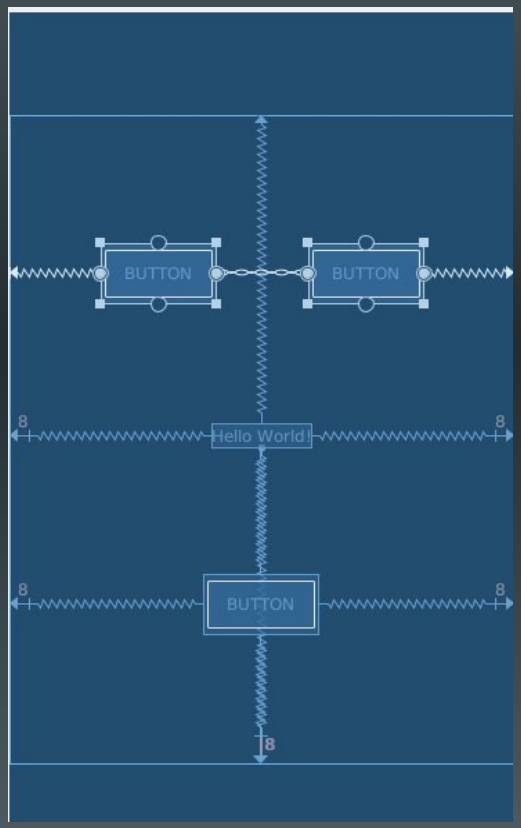






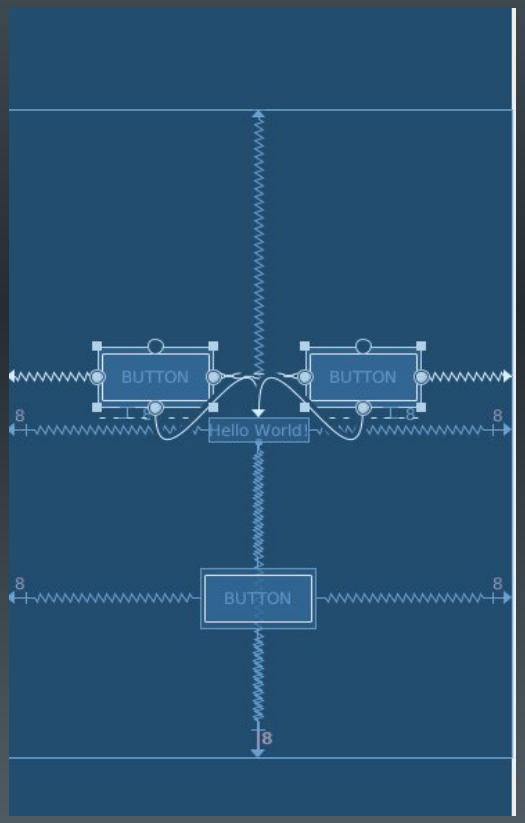
Pas de contraintes verticales
(y = 0)

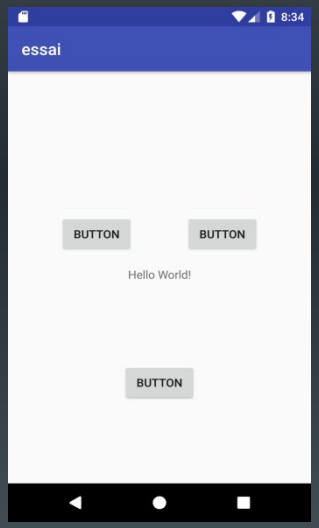
ConstraintLayout (6)

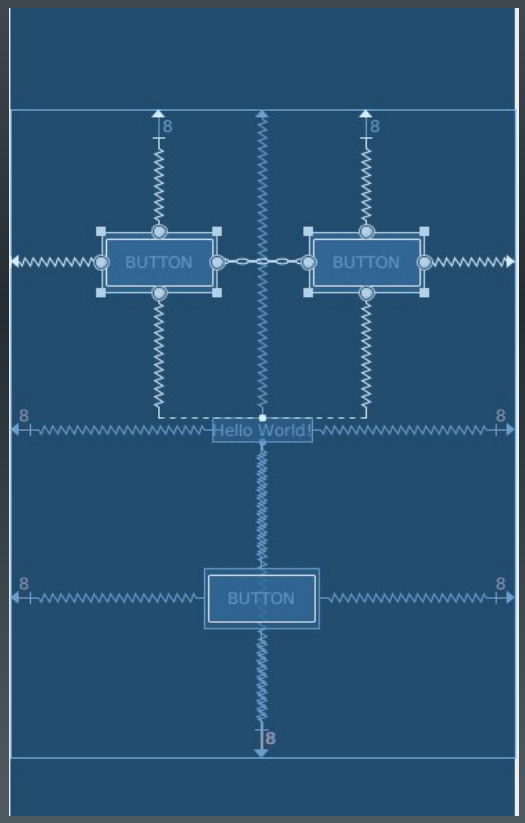


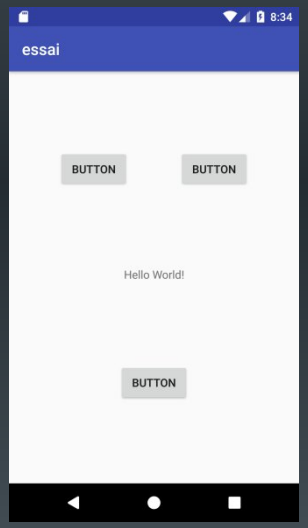

```
<Button
    android:id="@+id/button3"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Button"
    app:layout_constraintEnd_toStartOf="@+id/button4"
    app:layout_constraintHorizontal_bias="0.5"
    app:layout_constraintStart_toStartOf="parent"
    tools:layout_editor_absoluteY="101dp" />

  <Button
    android:id="@+id/button4"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Button"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.5"
    app:layout_constraintStart_toEndOf="@+id/button3"
    tools:layout_editor_absoluteY="101dp" />
```

ConstraintLayout (7)









Les Widgets

- Composants graphiques visibles par l'utilisateur
  - *Widgets* simples : zones de texte, boutons, listes, etc.
  - *Widgets* plus complexes : horloges, barres de progression, etc.
- Héritent de la classe View
- Utilisation :
  - Définition en XML (type, taille, centrage, position, etc.)
  - Comportement en Java
  - Peuvent également être créés dynamiquement en Java

Les TextView

- *Widget* permettant l'affichage d'un texte
  - Normalement non éditable
- Exemple :

```
<TextView
        android:id="@+id/text"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/letexte"
        android:hint="texte initial"
        android:layout_gravity="center"
        android:gravity="center"
/>
```

- Nombreux autres attributs
  - Cf classe TextView

Les EditText

- *Widget* permettant la saisie d'un texte (TextFields)
  - Accès : ouverture d'un clavier pour la saisie
  - nombreux attributs permettant l'aide à la saisie

```
<EditText
    android:id="@+id/email_address"
    android:layout_width="fill_parent"
    android:layout_height="wrap_content"
    android:hint="@string/email_hint"
    android:inputType="textEmailAddress" />
```

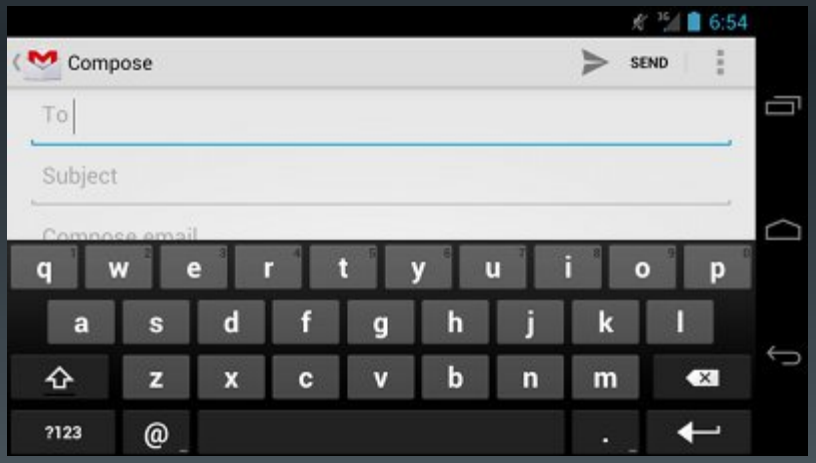
"text" : Normal text keyboard.

"textEmailAddress" : Normal text keyboard with the @ character.

"textUri" : Normal text keyboard with the / character.

"number" : Basic number keypad.

"phone" : Phone-style keypad.



Source : developer.android.com

Les Button

- *Widget* représentant un bouton d'action
  - Renvoie un événement lors de l'appui
  - Peut contenir un texte, une image ou les deux
- Exemples :

```
<Button
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/button_text"
    ... />
```

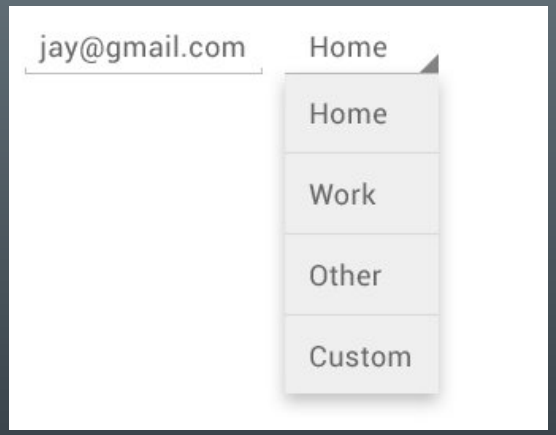
```
<Button
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/button_text"
    android:drawableLeft="@drawable/button_icon"
    ... />
```

```
<ImageButton
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:src="@drawable/button_icon"
    ... />
```

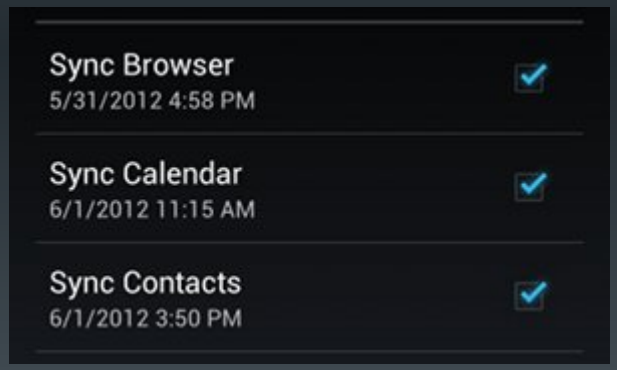
Source : developer.android.com

En vrac ...

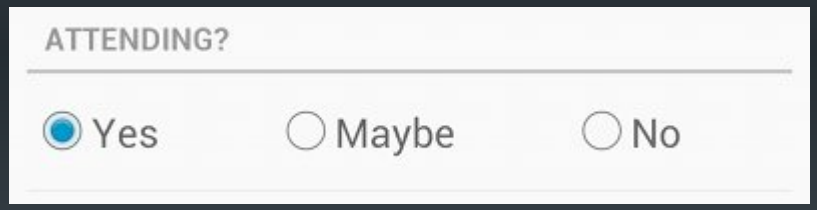
- Quelques autres *widgets*
  - Source developer.android.org



Spinner



CheckBox



RadioButton

ToggleButton

Switch
(android 4.0+)

Implantation du comportement (1)

- Les fichiers XML ne permettent que de :
  - positionner les composants ;
  - définir leurs caractéristiques.
- Nécessité de :
  - définir leur comportement
    - type d'interaction (clic court, clic long, etc.)
    - code de prise en compte (Java)
  - lier composant et code (XML ou Java)
    - XML : attribut android:onClick
    - Java : instancier un *event listener*

Implantation du comportement (2)

- Attribut android:onClick
  - Doit être suivi du nom de la méthode à appeler en cas de déclenchement
  - Prototype :
    - public void nomDeLaMethode(View maVue)

```
<Button
        android:id="@+id/monBouton"

    android:layout_width="wrap_content"

    android:layout_height="wrap_content"
        android:text="@string/monTexte"

    android:onClick="onBoutonClique"
/>
```

```
public void onBoutonClique(View maVue) {

    System.out.println("le bouton a été cliqué") ;

}
```

Permet de récupérer des informations sur le composant graphique qui a généré l'événement
Initialisé par le système avant l'appel

Récupération :

maVue.getId() ➡ R.id.monBouton

Implantation du comportement (3)

- Les *event listener*
  - interfaces de la classe View
  - ne disposent que d'une seule méthode à implanter
  - méthode appelée quand le composant associé est déclenché par l'utilisateur
- Exemples :

Interface	Méthode
View.OnClickListener	abstract void onClick(View v)
View.OnLongClickListener	abstract boolean onLongClick(View v)
View.OnFocusChangeListener	abstract void onFocusChange(View v, boolean hasFocus)

Implantation du comportement (3)

- Exemple : l'interface View.onClickListener
  - public void onClick(View v)

Retrouver un widget à partir du nom qui lui a été associé dans le xml

```
...
Button button = (Button) findViewById(R.id.button_name);

button.setOnClickListener(new View.OnClickListener() {

    public void onClick(View v) {
        // Do something in response to button click
    }

});
...
```

Création d'un « OnClickListener »

Source : developer.android.com

Surcharge de la méthode « onClick » de l'interface « onClickListener »

Associer un « onClickListener » au bouton

TP 02

Plan du cours

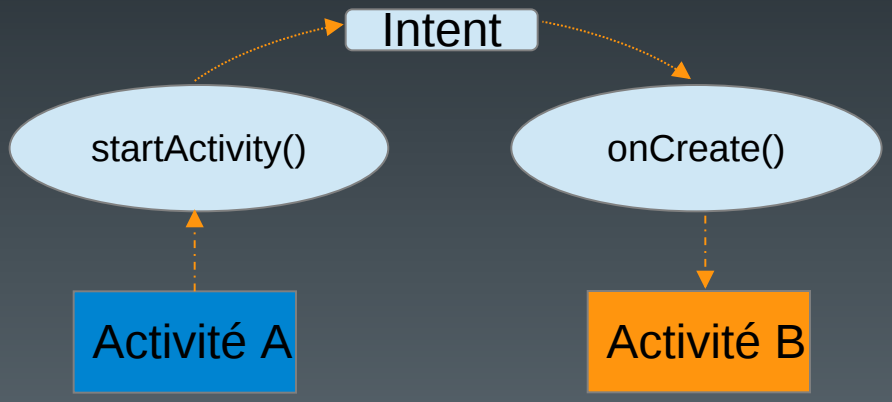
- Introduction
- Architecture d'une application Android
- Les activités
- Définir une interface graphique
- Les intentions explicites
- Les intentions implicites
- Les menus
- Les listes
- Les permissions
- Les content providers

Les Intentions

- Classe représentant un message échangé entre une activité et un composant présent sur le système
  - Une autre activité
  - Un service
  - Un diffuseur d'événements
- Deux types de messages
  - Explicite : on nomme le composant à démarrer
  - Implicite : on demande au système de trouver un composant adéquat, en fonction d'une action à effectuer

Les Intentions explicites

- Message adressé à un composant connu
  - On donne le nom de la classe correspondante
  - Généralement réservé aux composants appartenant à la même application …
- Le composant est démarré immédiatement



Création et lancement

Création d'une intention

```
Intent intention = new Intent (…) ;
…
startActivity(intention) ;
```

Démarrage d'une activité

```
Intent intention = new Intent (…) ;
…
startService(intention) ;
```

Démarrage d'un service

```
void maFonction(...) {
  ...
  Intent intention = new Intent (…) ;
  …
  startActivityForResult(intention, requestCode) ;
  …
}

protected void onActivityResult (int requestCode, int resultCode, Intent data){
  …
}
```

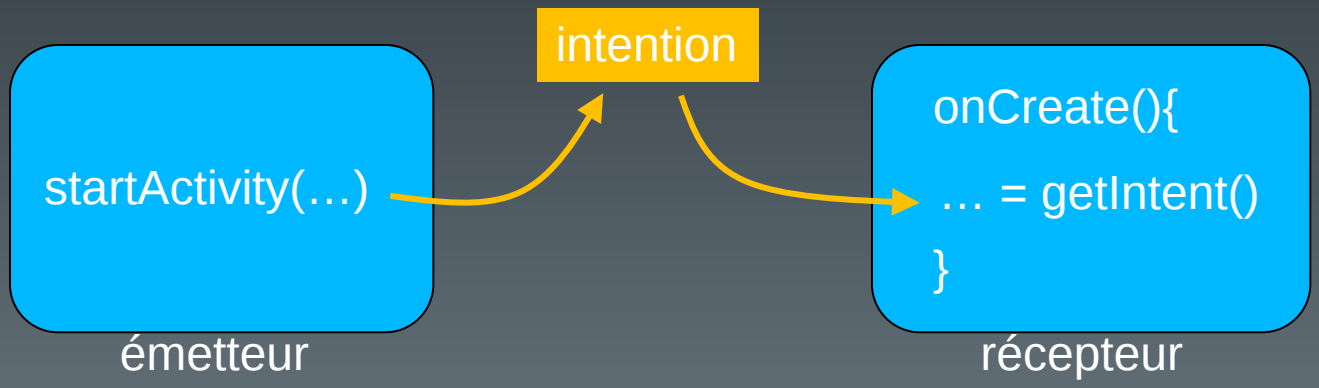
Déprécié depuis android 12

Démarrage d'une activité avec attente d'un résultat, récupéré dans la surcharge de cette méthode

Récupération

Code côté **récepteur**

```
public void onCreate (Bundle savedInstanceState) {
    ...
    // récupération de l'intention
    Intent intention = getIntent () ;
    …
    // extraction des informations reçues
    ...
    // traitement des informations reçues
    …
}
```



Intentions avec résultats (émetteur)

```
public static final int CODE = 4 ;
```

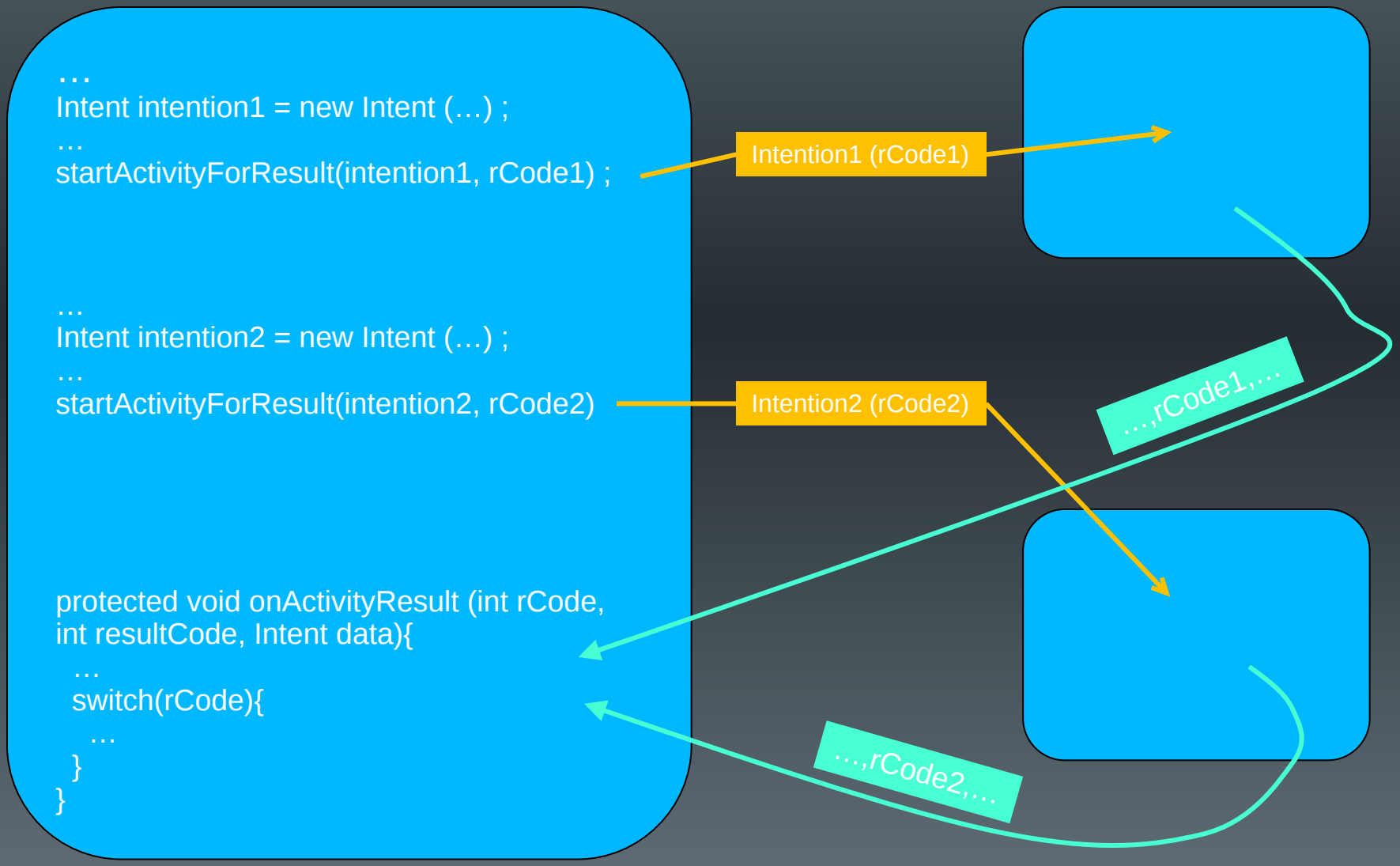
Code créé par le développeur pour identifier de manière unique son intention (>0)

```
void maFonction(...) {
  ...
  Intent intention = new Intent (…) ;
  …
  startActivityForResult(intention, requestCode) ;
  …
}


protected void onActivityResult (int requestCode, int resultCode, Intent data){
  …
}
```

Code de retour d'exécution de l'intent
Activity.RESULT_OK
Activity.RESULT_CANCELED

Données transmises en retour à l'activité appelante

Intentions avec résultats (émetteur)



Intentions avec résultats (récepteur)

```
void maFonction(...) {
  ...
  Intent intention = new Intent (…) ;
  …
  startActivityForResult(intention,
                         requestCode) ;
  …
}


protected void onActivityResult (int requestCode,
                                 int resultCode,
                                 Intent data){
  …
}
```

```
public void onCreate (Bundle savedInstanceState)
{
    // récupérer l'intent
    Intent intention = getIntent () ;

    // extraire les données et les traiter
    …
    // créer l'intent résultat
    Intent resultat = new Intent();
    // ajout des résultats
    …
    // prépare le retour des résultats
    setResult(RESULT_OK, resultat);
    finish();
}
```

ou

```
setResult(RESULT_CANCELED, resultat) ;
finish();
```

Remarques (1)

(1)

```
void maFonction(...) {
  ...
  Intent intention = new Intent (…) ;
  …
  startActivityForResult(intention,
                         requestCode) ;
  …
}

protected void onActivityResult (int requestCode,
                                 int resultCode,
                                 Intent data){
  …
}
```

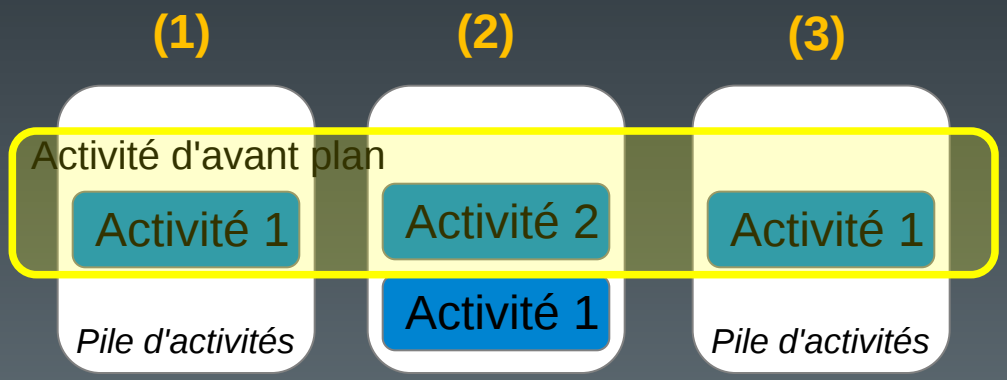
Activité1

(2)

(3)

```
public void onCreate (Bundle savedInstanceState)
{
  // récupérer l'intent
  Intent intention = getIntent () ;

  // extraire les données et les traiter
  …
  // créer l'intent résultat
  Intent resultat = new Intent();
  // ajout des résultats
  …
  // prépare le retour des résultats
  setResult(RESULT_OK, resultat);
  finish();
}
```

Activité2



Remarques (2)

(1)

```
public void onCreate (Bundle savedInstanceState) {
    // récupérer l'intent
    Intent intention = getIntent () ;
    …
}


void maFonction(...) {
  ...
  Intent intention = new Intent (Activité2) ;
  …
  startActivity(intention);
  …
}
```

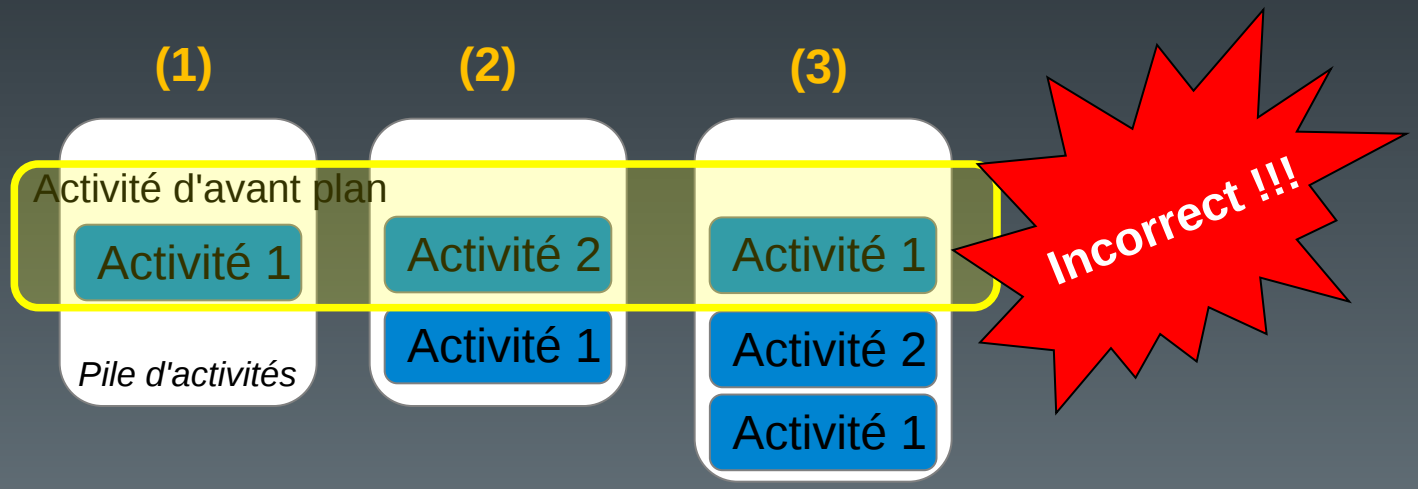
Activité1

(2)

(3)

```
public void onCreate (Bundle savedInstanceState)
{
  // récupérer l'intent
  Intent intention = getIntent () ;
  …

  Intent resultat = new Intent(Activité1);
  // ajout des résultats
  …
  // renvoyer les résultats
          startActivity(resultat);
}
```

Activité2



Intentions avec résultats (1)

- Modification du mécanisme à partir d'Android 12
- Côté émetteur :
  - Remplacement de startActivityForResult par registerForActivityResult
    - plus de fonction *callback* onActivityResult()
    - fonction *callback* définie au moment de la création de la classe de lancement
  - Quelques raisons
    - Résultats manqués si le composant de départ est recréé
    - Conflits avec le même code de requête
    - ...
- Côté récepteur :
  - Rien ne change

Intentions avec résultats (2)

- Plus de surcharge de onActivityResult()
  - Utilisation d'une instance de la classe ActivityResultLauncher<>
    - Template spécifiant le type de résultat attendu
    - Méthode de lancement launch()
  - Cas des intentions explicites

Ce que reçoit le lanceur

```
public class MainActivity extends AppCompatActivity {
    // attributs de la classe
    private ActivityResultLauncher<Intent> mGetResult = …

    protected void onCreate(Bundle savedInstanceState) { … }

    public void onClick(View v){
      Intent mess = new Intent(…);
      …
      mGetResult.launch(mess);
}
```

Intentions avec résultats (3)

- Création/enregistrement de la fonction callback
  - Classe registerForActivityResult()
    - Deux paramètres au constructeur :
      - Type d'activité (classe de contrat)
        - Type de données en entrée
        - Type de données en sortie
        - Nombreuses classes disponibles
      - La fonction callback à utiliser
    - Enregistre la fonction callback
    - Retourne un lanceur (ActivityResultLauncher)
      - Un lanceur par activité "fille"
        - Pas de requestCode

Intentions avec résultats (4)

– Cas des intentions explicites

```
public class MainActivity extends AppCompatActivity {
    // attributs de la classe
    private ActivityResultLauncher<Intent> mGetResult = registerForActivityResult(

            // classe de contrat pour une intention explicite
            new ActivityResultContracts.StartActivityForResult(),

            // callback pour une intention explicite
            new ActivityResultCallback<ActivityResult>() {
                @Override
                public void onActivityResult(ActivityResult result) {

                }// onActivityResult

            }// ActivityResultCallback

        );// registerForActivityResult


    protected void onCreate(Bundle savedInstanceState) {
        …
    }

  …
}
```

Ce que récupère la fonction callback

Intentions avec résultats (5)

- Cas des intentions explicites
  - Récupérer les résultats

```
private ActivityResultLauncher<Intent> mGetResult = registerForActivityResult(

        // classe de contrat pour une intention explicite
        new ActivityResultContracts.StartActivityForResult(),

        // callback pour une intention explicite
        new ActivityResultCallback<ActivityResult>() {
            @Override
            public void onActivityResult(ActivityResult result) {

                if(result.getResultCode() == RESULT_OK){
                  Intent mess = result.getData();
                  …
                }

            }// onActivityResult

        }// ActivityResultCallback

);// registerForActivityResult
```

Structure générale des Intentions

Nom du composant de destination
Optionnel : utilisé uniquement pour les intentions explicites

intentions explicites

Intent

composant	action	données	catégorie	type	extras	flags

intentions implicites

Extras
contenu supplémentaire transitant entre les composants

Indicateurs
permettant de modifier le comportement du cycle de vie des activités

Spécifier le nom du composant

- Différentes méthodes utilisables :
  - Intent(...) : le constructeur
  - SetComponent(...), setClassName(...), setClass(...), etc.

```
...
Intent = new Intent(this, SecondActivity.class) ;
...
startActivity(intent) ;
...
```

La classe

Le contexte

Le package

```
...
Intent = new Intent() ;
Intent.setClassName(this, "fr.univ-littoral.renaud.bidon.SecondActivity") ;
...
startActivity(intent) ;
...
```

```
...
Intent = new Intent() ;
Intent.setComponent(new ComponentName("fr.univ-littoral.renaud.bidon",
                                      "fr.univ-littoral.renaud.bidon.SecondActivity") ;
...
startActivity(intent) ;
...
```

Les extras (1)

- Permettent d'insérer des données dans l'intention
  - Communications inter-composants
  - Structure : (clé, valeur)

Identifiant de la donnée

Valeur de la donnée

type = n'importe quel type de base (boolean, int, String, float[], ...)

insertion

Intent.putExtra(String cle, *type* valeur) ;

- Clé définie par l'utilisateur

public final static String MACLE = "fr.ulco.renaud.appli.MACLE" ;

- Quelques clés prédéfinies

récupération

type get{*type*}Extra(String cle, *type* vdefaut) ;

- vdefaut : valeur par défaut renvoyée si la clé n'est pas trouvée dans l'intention
- pas de valeur par défaut pour les tableaux, ni les String

type[] get{Type}ArrayExtra(String cle) ;
String getStringExtra(String cle) ;

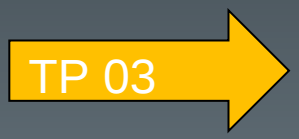
Les extras (2)

```
public class mainActivity extends Activity {

  public final static String MARQUE = "fr.ulco.renaud.appli.MARQUE" ;
  public final static String PUISS = "fr.ulco.renaud.appli.PUISS" ;

…
  Intent intention = new Intent(this, otherActivity.class) ;
  String marque = "Citroen" ;
  int puissance = 6 ;
  intention.putExtra(MARQUE, marque) ;
  intention.putExtra(PUISS, puissance) ;
  startActivity(intention) ;
…
}
```

```
public class otherActivity extends Activity{
…
  Intent intention = getIntent() ;
  int p = intention.getIntExtra(mainActivity.PUISS, 0) ;
  String m = intention.getStringExtra(mainActivity.MARQUE) ;
…
}
```

Les extras (3)

- Possibilité d'ajouter un Bundle :
  - Intent.putExtras(String cle, Bundle valeur) ;
  - Bundle getBundleExtras(String cle) ;
- Possibilité d'ajouter une intention :
  - Intent.putExtras(Intent valeur) ;
  - Recopie tous les extras de « valeur » dans l'intent appelant
- Possibilité d'ajouter des objets complexes :
  - Doivent être sérialisables
    - La classe d'origine doit implémenter l'interface Parcelable
    - Intent.putExtra(String cle, Classe valeur) ;
    - Classe getParcelableExtra(String cle) ;

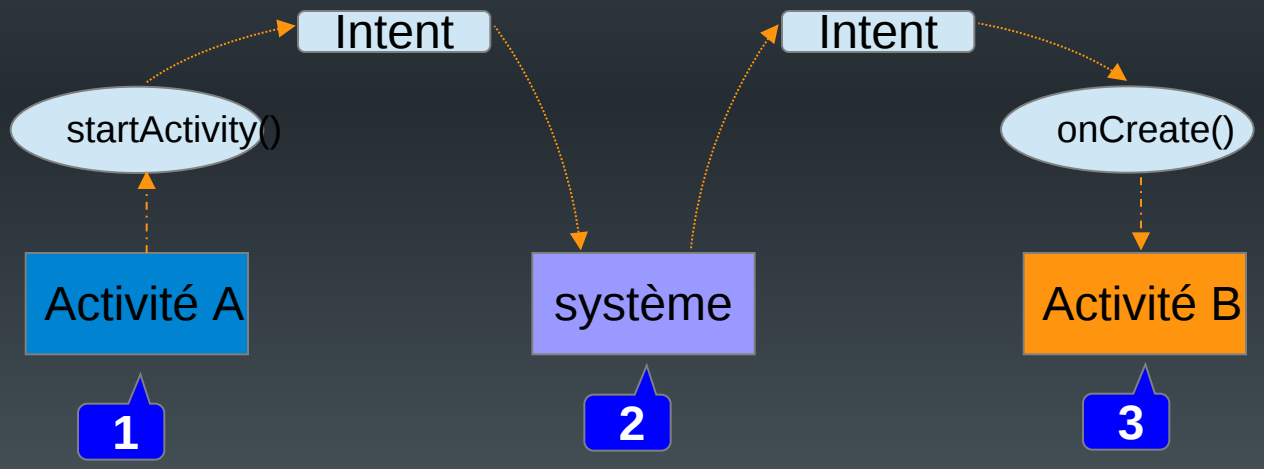


Plan du cours

- Introduction
- Architecture d'une application Android
- Les activités
- Définir une interface graphique
- Les intentions explicites
- Les intentions implicites
- Les menus
- Les listes
- Les permissions
- Les content providers

Les Intentions implicites

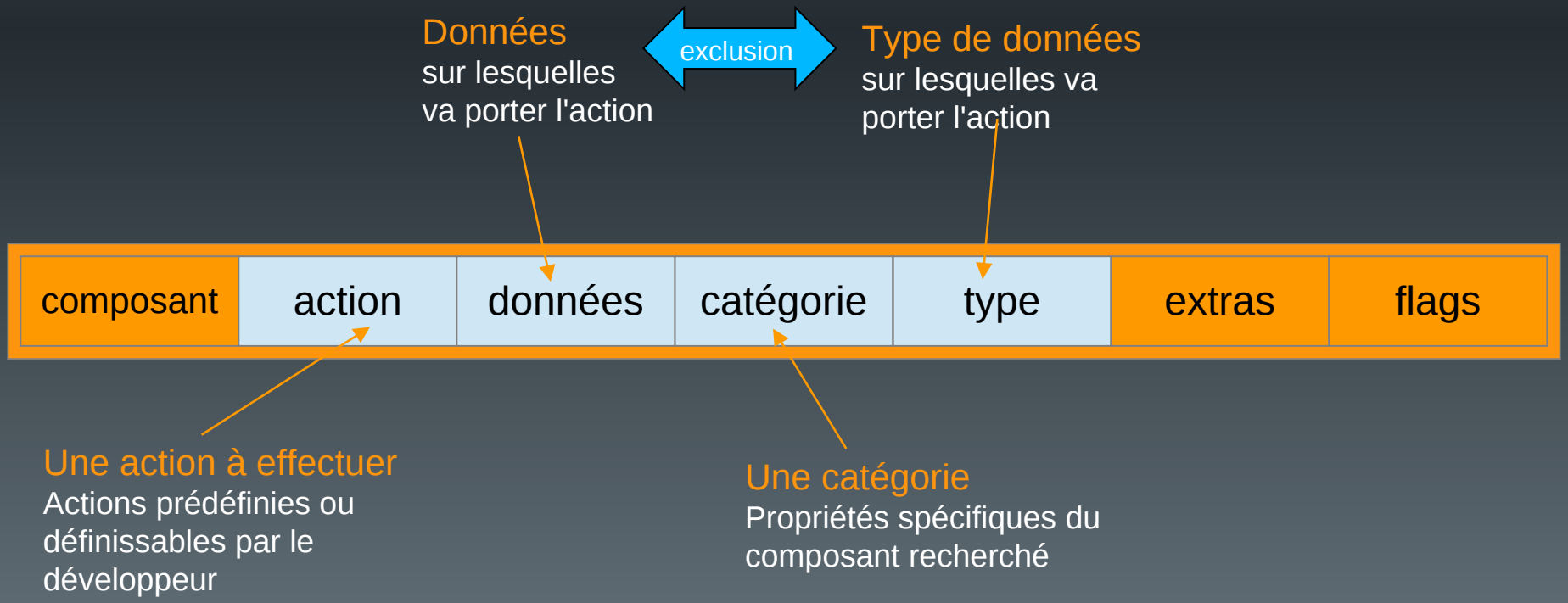
- Message à destination d'un composant inconnu
  - Le système se charge de trouver le composant adéquat et le démarre



  - En cas de possibilités multiples, affichage des choix à l'utilisateur

Les Intentions implicites

- Message à destination d'un composant inconnu
  - On ne peut pas préciser son nom
  - Utilisation de méta-données permettant au système de sélectionner les applications utilisables



Action à effectuer

composant	action	données	catégorie	type	extras	flags

- Nombreuses constantes prédéfinies
  - Dans la classe Intent :
    - ACTION_MAIN : démarre l'activité comme point d'entrée principal
    - ACTION_VIEW : affiche les données fournies à l'utilisateur
    - ACTION_EDIT : permet l'édition des données fournies par l'utilisateur
    - ACTION_SEND : permet l'envoi des données (mail, réseau social, ...)
    - etc
  - Exemples :

```
Intent intention = new Intent();
intention.setAction(Intent.ACTION_EDIT);
```

```
Intent intention = new Intent(Intent.ACTION_EDIT, ...);
```

  - Possibilité de définir ses propres actions
    - Comme constante de classe

```
static final String ACTION_BIDON = "fr.univ-littoral.renaud.bidon.BIDON";
```

type — Nom — Valeur, incluant le nom du package

Catégorie

- Précise le type de composant qui doit prendre en charge l'intention
- Exemples :
  - CATEGORY_LAUNCHER : le composant doit être visible dans le lanceur d'applications
  - CATEGORY_CAR_MODE : le composant doit être utilisable en mode conduite de véhicule
- Remarques
  - Nombre quelconque de catégories dans une intention
  - La plupart des intentions ne nécessitent pas de catégories

```
Intent intention = new Intent();
intention.addCategory(Intent.CATEGORY_CAR_MODE);
intention.addCategory(Intent.CATEGORY_LAUNCHER);
```

Données

- Formatées à l'aide des URI (*Uniform Ressource Identifier*)
  - Chaîne de caractères représentant un endroit ou une ressource
  - Syntaxe normalisée :

Précision quant à l'information

<schéma> : <information> { ? <requêtes> } { # <fragment> }

Accès à une sous-partie de l'information

Nature de l'information :

http, https
content
geo
file
tel , voicemail
sms,smsto, mms, mmsto

L'information :
- Dépend du schéma
- Exemples :

tel:06000007
geo:123.456,12.3456
http://www.google.fr

```
Intent intention = new Intent() ;
Uri sms = Uri.parse("sms:06000007") ;
intention.setData(sms) ;
```

Le type

- Normalement implicite dans les données
- Absence de données/recherche plus précise :
  - peut être précisé en utilisant les codes MIME
    - (*Multipurpose Internet Mail Extensions*)
    - https://www.iana.org/assignments/media-types/media-types.xhtml
  - text, audio, video, application, ...
- Et des sous-types
  - text/plain, text/xml, text/html, ...
  - audio/mp3, audio/ogg, audio/wav, ...
  - image/jpeg, image/png, ...
  - video/mpeg, video/mp4, ...

```
Intent intention = new Intent() ;
…
intention.setType("audio/mp3") ;
…
String mime = intention.getType() ;
```

Élimine les données …
(setData élimine le type)

Lancement (sans résultats)

- Similaire aux intentions explicites :

```
Intent intention = new Intent();
Intention.setAction(Intent.ACTION_VIEW);
Intention.setType(""audio/*");
startActivity(intention);
```

- Gérer l'absence de composants adéquats :

```
Intent intent = new Intent() ;
...
if (intent.resolveActivity(getPackageManager()) != null) {
  startActivity(intent);
}else{
  ...
}
```

Utilisation du gestionnaire de paquets

```
Intent intent = new Intent() ;
...
try {
    startActivity(intent);
} catch (ActivityNotFoundException e) {
    // aucune activité ne répond à la demande
}
```

Utilisation des exceptions

Lancement (avec résultats) (1)

- Par startActivityForResult (déprécié)
  - Identique aux activités explicites
  - Récupération des résultats dans le champ "données" de l'intention résultats
    - On récupére un « lien » vers les données

```
Uri uri = data.getData();
```

Lancement (avec résultats) (2)

- Par classe de contrat (à partir de Android 12)

Type de ce qu'on l'on va envoyer

Dépend de la classe de contrat

```
public class MainActivity extends AppCompatActivity {
    // attributs de la classe
    private ActivityResultLauncher<TYPE> mGetResult = registerForActivityResult(

            // classe de contrat pour une intention implicite
            new ActivityResultContracts.XXX(),

            // callback pour une intention implicite
            new ActivityResultCallback<TYPE_RESULTAT>() {
                @Override
                public void onActivityResult(TYPE_RESULTAT result) {

                }// onActivityResult

            }// ActivityResultCallback

        );// registerForActivityResult


}
```

Type de ce qu'on l'on va récupérer

Lancement (avec résultats) (3)

- Nombreuses classes de contrat
  - ActivityResultContracts.StartActivityForResult
    - Cf activités explicites
  - ActivityResultContracts.CaptureVideo
    - Retourne l'uri d'une vidéo faite par l'utilisateur
  - ActivityResultContracts.GetContent
    - Retourne une uri d'une donnée choisie par l'utilisateur
  - ActivityResultContracts.PickContact
    - Retourne un contact choisi par l'utilisateur
  - ...

Lancement (avec résultats) (4)

- Un exemple :
  - Choisir un fichier stocké sur le périphérique (image, son, vidéo, etc...)
- Quelle classe de contrat ?
  - ActivityResultContracts.GetContent

myLauncher.launch("audio/*")

Qu'envoie-t-on ?

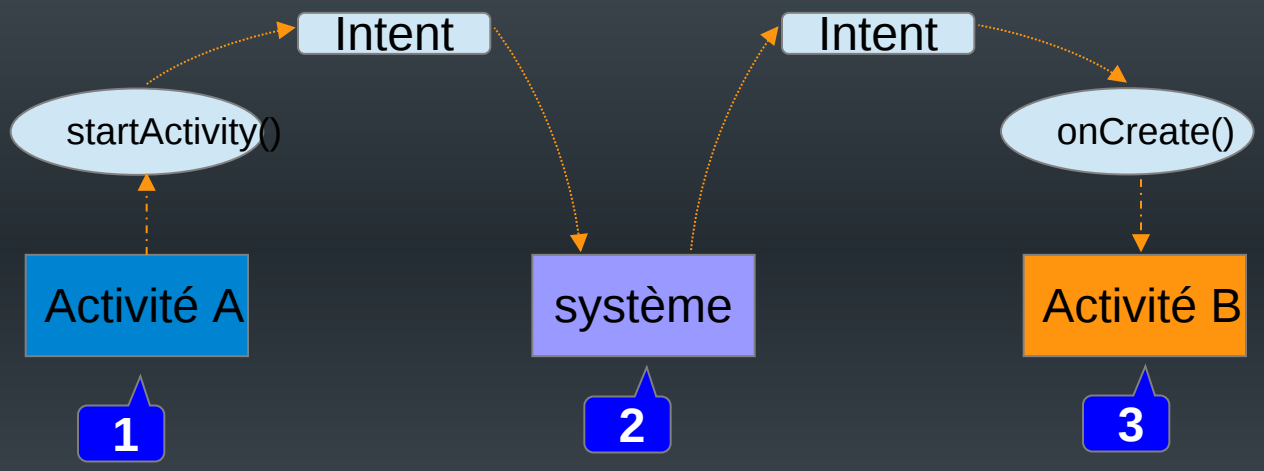
le type mime du fichier

```
ActivityResultLauncher< String > myLauncher = registerForActivityResult(
        new ActivityResultContracts.GetContent(),
        new ActivityResultCallback< Uri >() {
            @Override
            public void onActivityResult( Uri resultat ){
                ...
            }
        });
```

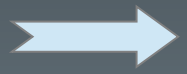
Que récupère t-on ?

Recherche d'un composant

- Utilisation d'intentions implicites :
  - Le système recherche le(s) composant(s) possible(s)



  - Le système doit connaître les types d'intentions que peut recevoir chaque composant
    - Nécessité de préciser pour chaque composant les intentions réceptionnables



Intent filters

Les intent filters

- Présents dans le fichier *manifest.xml* pour chaque composant d'une application
- Syntaxe :

activity,
...

```
<composant ...>
    <intent-filter>
        <action android:name="..." />
        <category android:name="..." />
        <data android:mimeType="..." />
    </intent-filter>
</composant>
```

Peuvent être en nombre quelconque

Filtre d'intentions

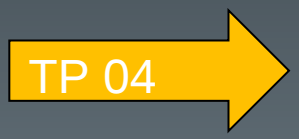
Composant principal d'une application

```
<intent-filter>
    <action android:name="android.intent.action.MAIN" />
    <category android:name="android.intent.category.LAUNCHER" />
</intent-filter>
```

Doit apparaître dans la liste des applications pouvant être lancées

Remarques

- Intentions et services :
  - Démarrage d'un service :
    - N'utiliser que des intentions explicites
    - Intentions implicites : l'utilisateur ne sait pas forcément quel service est lancé et il n'a pas de retour visuel
  - Pas d'*intent filter* pour un service



Plan du cours

- Introduction
- Architecture d'une application Android
- Les activités
- Définir une interface graphique
- Les intentions explicites
- Les intentions implicites
- Les menus
- Les listes
- Les permissions
- Les content providers

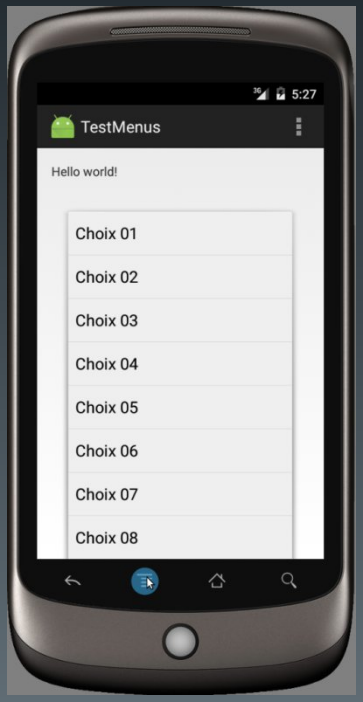
Les menus

- Disponibles pour chaque activité
- Trois types de menus :
  - Le menu d'options
  - Les menus contextuels
  - Les menus pop-ups
- Considérés comme des ressources
  - À définir en XML (dossier res/menus)
    - Facilite la maintenance des menus
      - séparation de la définition du code de gestion
    - Possibilité de créer des menus multi-plateforme
    - Facilite la visualisation de la structure des menus

Possibilité de les définir dans le code Java

Le menu d'options (1)

- Accessibles via :
  - le bouton de « menu »
    - N'est plus forcément disponible sur les périphériques récents
  - l'icône « menu » de la barre d'application



Le menu d'options (2)

- Définition XML

```
<?xml version="1.0" encoding="utf-8"?>
<menu   xmlns:android="http://schemas.android.com/apk/res/android"
        xmlns:tools="http://schemas.android.com/tools"
        tools:context=".MainActivity">


  <item android:id="@+id/choix01"
        android:title="@string/texte_choix01" />

    ...

  <item android:id="@+id/choixNN"
        android:title="@string/texte_choixNN" />

</menu>
```

Identifiant de chaque entrée

Libellé du choix

Le menu d'options (3)

- Attributs des items

```
<item android:id="@+id/choix01"
        android:title="@string/texte_choix01"
        android:icon="..."
        android:onClick="..."
        android:showAsAction=" "
/>
```

Icône associée au choix

Callback en cas de choix

Précise quand et comment l'item peut apparaître dans la barre d'application :
•"ifRoom" : s'il y a de la place
•"never" : jamais
•"always" : toujours (à éviter sauf item important pour des raisons de place)
•…

Le menu d'options (4)

- Chargement du menu
  - Appel de la méthode *onCreateOptionsMenu()* au lancement de l'application

Menu de l'application devant être initialisé

```
@Override
public boolean onCreateOptionsMenu(Menu menu) {
    // Inflate the menu; this adds items to the action bar if it is present.
        MenuInflater inflater = getMenuInflater();
        inflater.inflate(R.menu.monMenu, menu);

    return true;
}
```

Objet permettant de gérer l'association XML/menu

Association XML/menu

Le menu d'options (5)

- Gestion des choix
  - Appel de la méthode *onOptionsItemSelected()* à chaque sélection (par défaut)

L'item choisi

```
@Override
public boolean onOptionsItemSelected(MenuItem item) {

        int id = item.getItemId();

        if(id==R.id.choix01) { ... }
        else if(id==R.id.choix02) { ... }
        else ...

}
```

Récupération de l'identifiant de l'item dans la classe R

  - Remplaçable par une méthode *nom(MenuItem item)*

```
public boolean nom(MenuItem item){ … }   <item …
                                          android:onClick="nom" />
```

Le menu d'options (6)

- Extensions aux sous-menus
  - tout item peut contenir un sous-menu

```
<?xml version="1.0" encoding="utf-8"?>
<menu   … ">

  <item …  />

  <item android:id="@+id/choixNN"
        android:title="@string/texte_choixNN" >
        <menu>
            <item … />
            <item … />
        </menu>
  </item>

  <item … />

</menu>
```

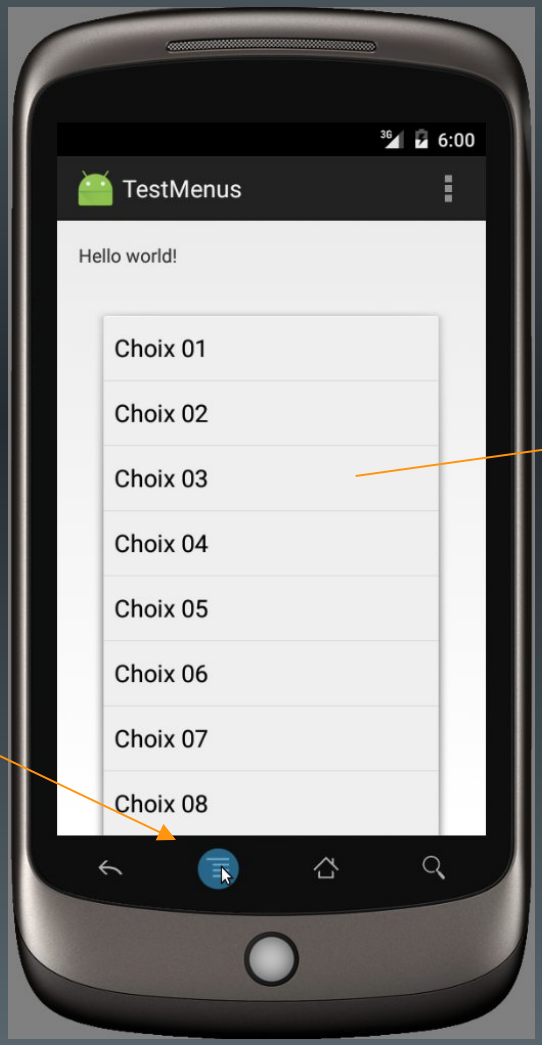
Menu principal

Items du menu principal

Sous menu

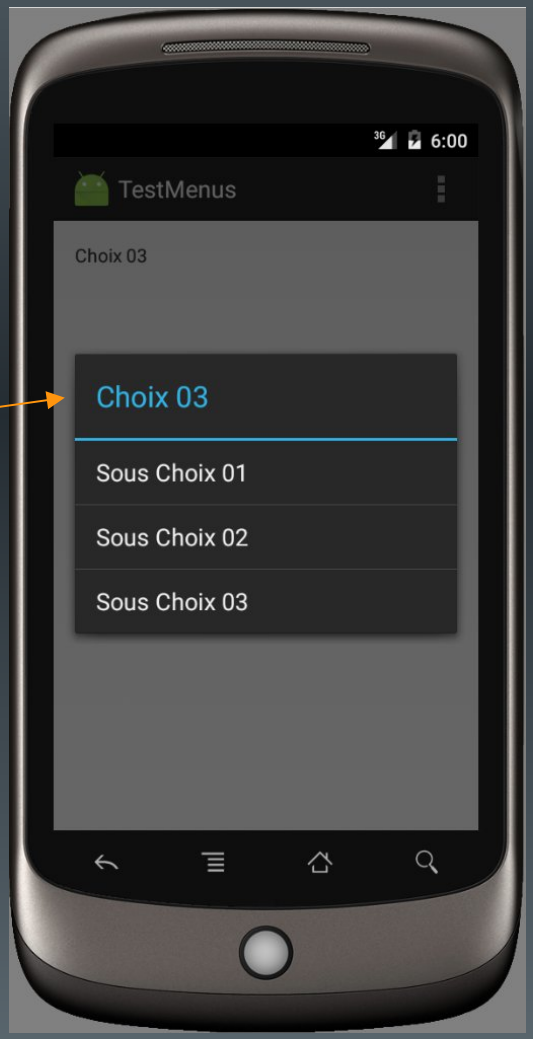
Le menu d'options (7)

- Exemples



Sélection du menu principal

Sélection d'un sous-menu



Le menu d'options (8)

- Remarque sous AndroidStudio
  - Par défaut : pas de barre d’action ...

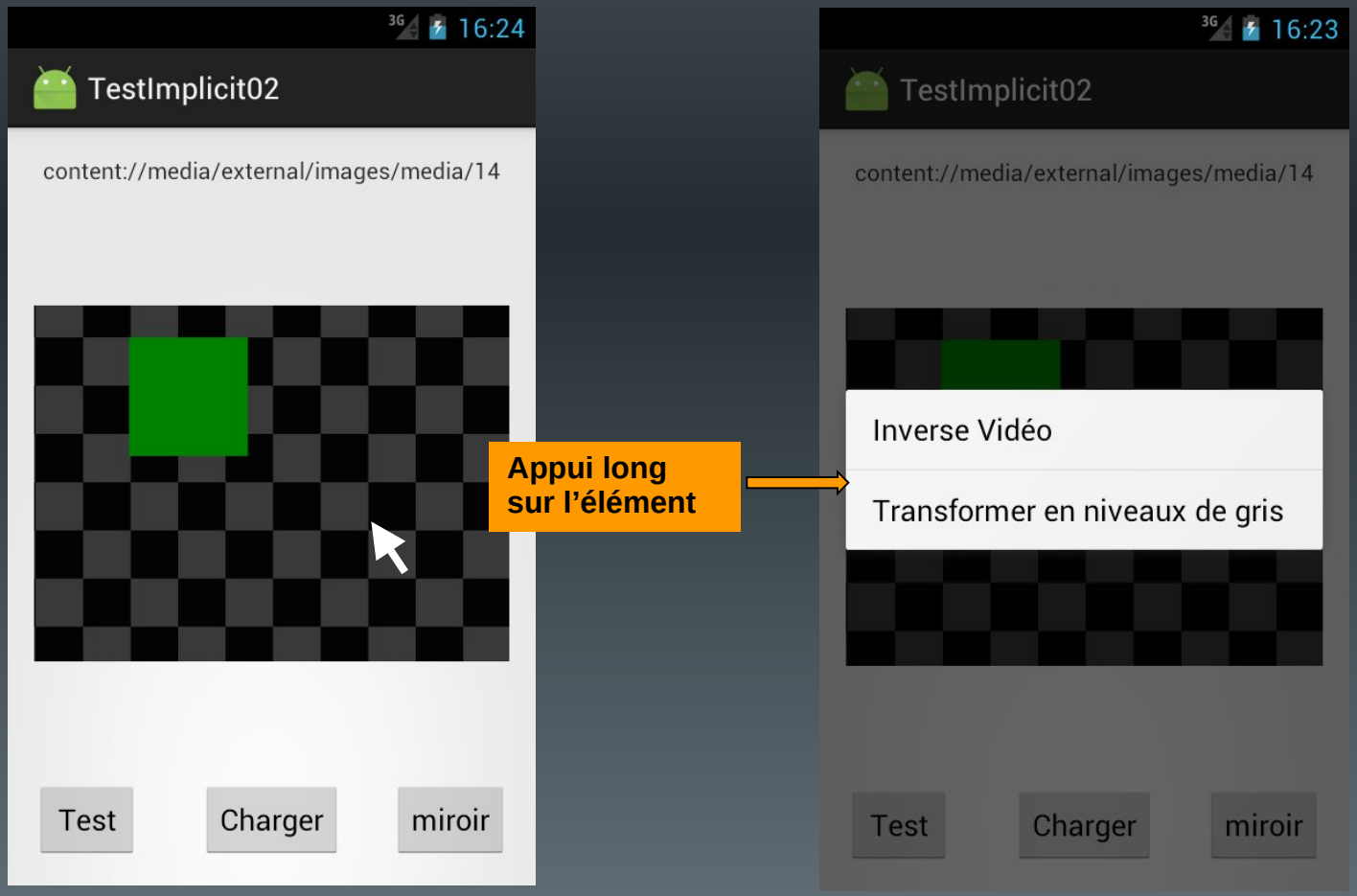
fichier de ressources : res/values/themes/themes/xml

```
<resources ... >
   <style name="Base.Theme.MonAppli"
            parent="Theme.Material3.DayNight.NoActionBar">
      ...
   </style>
</resources ... >
```

```
<resources ... >
   <style name="Base.Theme.MonAppli"
            parent="Theme.Material3.DayNight">
      ...
   </style>
</resources ... >
```

Les menus contextuels (1)

- Associables à tout élément de l'interface graphique



Les menus contextuels (2)

- Utilisent un fichier XML décrivant un menu

1. Enregistrer l'élément pour lequel un menu contextuel doit être créé

```
registerForContextMenu(View view)
```

2. Surcharger la méthode onCreateContextMenu()

```
@Override
public void onCreateContextMenu(ContextMenu menu, View v,
                                ContextMenuInfo menuInfo) {

    super.onCreateContextMenu(menu, v, menuInfo);

    MenuInflater inflater = getMenuInflater();
    inflater.inflate(R.menu.context_menu, menu);
}
```

Nécessaire si plusieurs View utilisent un menu contextuel …

Informations complémentaires

Ex. item d'une liste ayant activé le menu

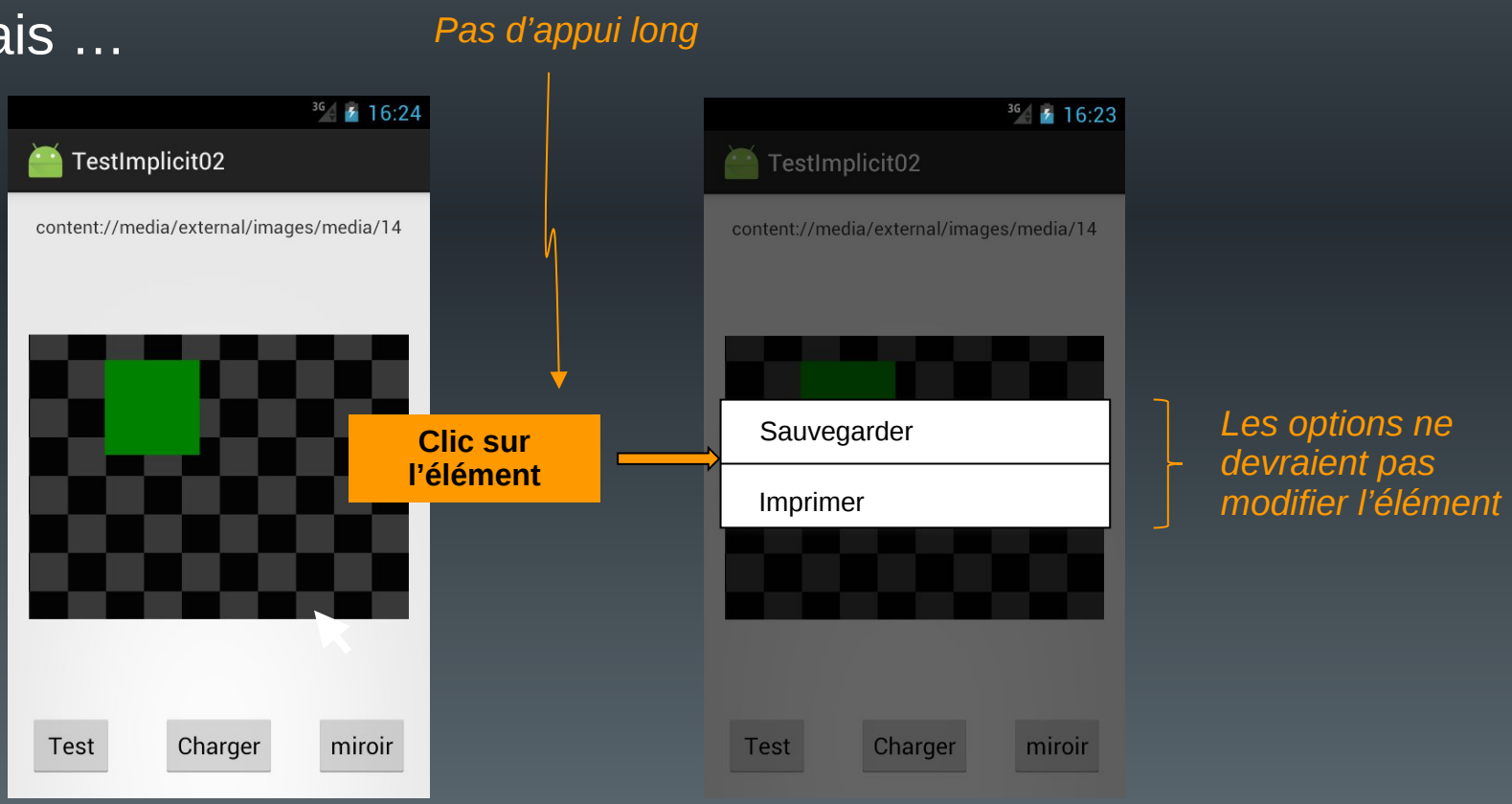
Les menus contextuels (3)

3. Surcharger la méthode onContextItemSelected()

```
@Override
public boolean onContextItemSelected(MenuItem item) {

    if (item.getItemId()==R.id.xxx) {
        ...
        return true;
    } else if (item.getItemId()==R.id.yyy) {
        ...
        return true;
    } else
        return super.onContextItemSelected(item);
    }

}
```

Les menus popup (1)

- Associables à tout élément de l'interface graphique
- Très similaires aux menus contextuels
  - Mais …



Gestion « manuelle » de l'événement …

Les menus popup (2)

(1) Créer le menu (XML)

```
<menu xmlns:android="http://schemas.android.com/apk/res/android">
        <item  android:id="@+id/popup01"
               android:title="@string/menu_popup01" />
        <item  android:id="@+id/popup02“
               android:title="@string/menu_popup02" />
 </menu>
```

(2) Associer une méthode de gestion du clic au widget

```
< Widget
        android:id="@+id/name"
        …

        android:onClick="showPopupX"
/>
```

(3) Créer la méthode d’ouverture du popup

```
public void showPopupX(View v) {
        PopupMenu popup = new PopupMenu(this, v);
        MenuInflater inflater = popup.getMenuInflater();
        inflater.inflate(R.menu.menu_popup, popup.getMenu());
        popup.inflate(R.menu.menu_popup);

        popup.setOnMenuItemClickListener(new PopupMenu.OnMenuItemClickListener(){
                @Override
                public boolean onMenuItemClick(MenuItem item) {
                        if(item.getItemId()== R.id.popup01) {
                        else
                           ...
                        }
                }
         });
        popup.show();
}
```

Créer le popup

Implémenter l’interface OnMenuItemClickListener

Associer un gestionnaire de clic

Afficher le popup

TP 05

Plan du cours

- Introduction
- Architecture d'une application Android
- Les activités
- Définir une interface graphique
- Les intentions explicites
- Les intentions implicites
- Les menus
- Les listes
- Les permissions
- Les content providers

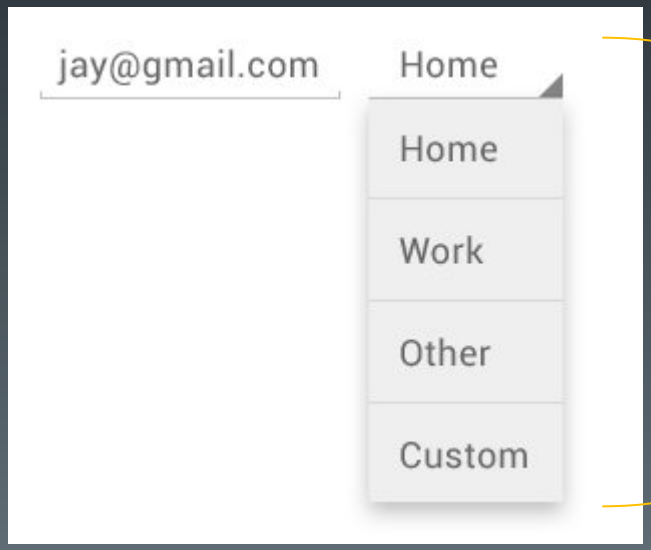
Les listes

- Éléments très utilisés sous android
  - Facilitent l'affichage et la manipulation d'ensembles de données importants sur des écrans de taille réduite
  - Possibilité de faire défiler/sélectionner les items
  - Paramétrage possible de la présentation des items via un layout
  - Remplissage « automatique » à partir de données extraites de tableaux ou de bases de données
    - Nécessitent l’utilisation d'un **adaptateur**

Exemples

ListView

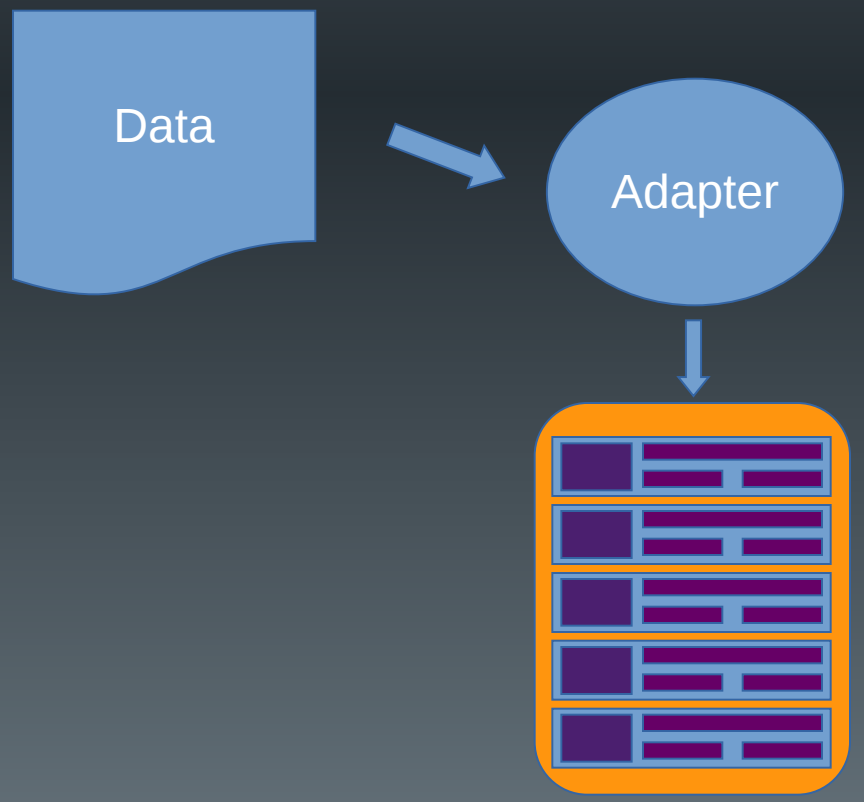
GridView



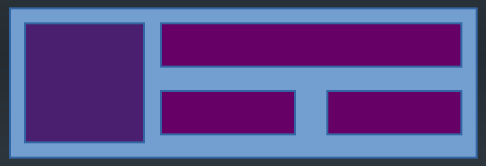
Spinner

Les adaptateurs (1)

- Rôle
  - Adapter les données au format d'affichage
  - Nécessité de préciser le layout utilisé pour les items



mise en forme (*layout*)



Layout prédéfinis :
android.R.layout.XXX

simple_list_item1
simple_list_item_checked
two_lines_list_items
...

Définir ses propres *layouts*
fichiers xml

Les adaptateurs (2)

- Différentes classes prédéfinies
  - ArrayAdapter : utilise un tableau d'objets quelconques ou une liste java
  - SimpleCursorAdapter : utilise des ensembles de données issus de requêtes sur des bases de données
  - SimpleAdapter : utilise des maps {(clé, valeur),…}

Le contexte de l'activité (this, getContext())

```
ArrayAdapter(Context c, int ressource) ;
ArrayAdapter(Context c, int ressource, T[ ] objets) ;
```

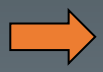
L'identifiant du fichier ressource décrivant le layout à utiliser (R.layout….)

Les objets à insérer dans une liste via l'adaptateur

Les adaptateurs (3)

- Quelques méthodes utiles (ArrayAdapter)
  - int getCount() : nb d'objets gérés par l'adaptateur
  - add(T objet) : ajoute l'objet à la fin de la liste
  - insert(T objet, int position) : insertion de l'objet à une position donnée
  - remove(T objet) : suppression de l'objet de la liste
  - clear() : suppression de tous les objets de la liste
  - getItem(int position)
- Remarque :

ArrayAdapter(Context c, int ressource, T[ ] objets) ;



Les objets insérés deviennent "*immutable*"
impossible d'en ajouter ou d'en supprimer ...

Les ListView (1)

- Composant graphique qui gère l'affichage d'une liste
  - Nécessite un adaptateur
- Configuration XML

```
<ListView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:id="@+id/maListe"
    android:choiceMode="none" />
```

none : éléments non sélectionnables
singleChoice : 1 seul choix possible
multipleChoice : choix multiples

L'adaptateur doit utiliser le layout correspondant ...

- Déclaration Java

```
ListView lv = (ListView)findViewById(R.id.maListe);
```

- Remplissage :
  - Statique : depuis un tableau de données xml
  - Dynamique : via un adaptateur

Les ListView (2)

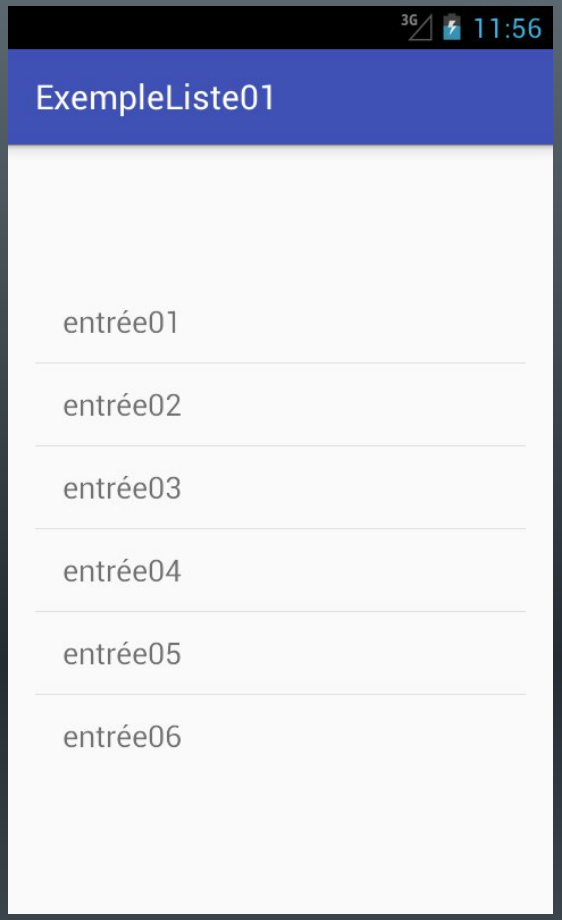
- Remplissage statique (xml)

res/values/strings.xml

```
<resources>
  ...
   <string-array name="uneListe">
      <item>entrée01</item>
      <item>entrée02</item>
      ....
      <item>entrée06</item>
   </string-array>
  ...
</resources>
```

res/layout/content.xml

```
<ConstraintLayout … >
   ...
   <ListView
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:id="@+id/maListe"
      android:entries="@array/uneListe" />
   …
</ConstraintLayout>
```

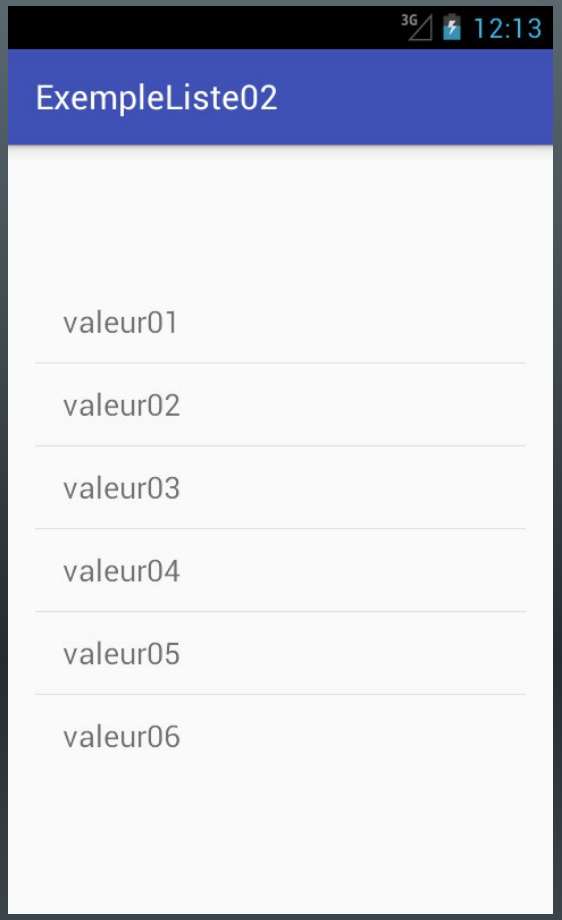


Les ListView (3)

- Remplissage dynamique (java)

res/layout/content.xml

```
...
<ListView
    ...
    android:id="@+id/maListe" />
...
```




```
ListView lv = (ListView)findViewById(R.id.maListe);

String entrees[]={"valeur01", "valeur02", "valeur03",
                  "valeur04", "valeur05", "valeur06"};

ArrayAdapter<String> adapter =
    new ArrayAdapter<String>(this,
                             android.R.layout.simple_list_item_1,
                             entrees);
lv.setAdapter(adapter);
```

TP 06

Plan du cours

- Introduction
- Architecture d'une application Android
- Les activités
- Définir une interface graphique
- Les intentions explicites
- Les intentions implicites
- Les menus
- Les listes
- Les permissions
- Les content providers

Les permissions (1)

- Restreindre ce qu'une application a le droit de faire
- Protéger les données présentes sur le périphérique
- Par défaut, impossibilité d'impacter :
  - Une autre application
  - Le système
  - Les données utilisateur
- Chaque application est lancée dans son propre espace (*sandbox*)
  - Nécessité de préciser les permissions
  - Validation par l'utilisateur

Les permissions (2)

- Deux groupes de permissions
  - Normales :
    - Sans risque pour l'utilisateur
    - ex. allumer le flash
    - Sont attribuées automatiquement sur demande de l'application
  - Dangereuses
    - Possibilité de compromettre les données utilisateur ou d'autres applications
    - ex. lire les contacts
    - Nécessitent une validation de l'utilisateur

Les permissions (3)

- Définir les permissions :
  - Où ?
    - Dans le fichier manifest.xml
  - Comment ?
    - Balise <uses-permission>

À l'extérieur de la balise <application>

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
   package="fr.ulco.renaud.myapp" >
    ...
   <uses-permission android:name="android.permission.ACCESS_WIFI_STATE" />
   <uses-permission android:name="android.permission.RECEIVE_SMS" />
   ...
   <application ...>
   ...
   </application ...>

</manifest>
```

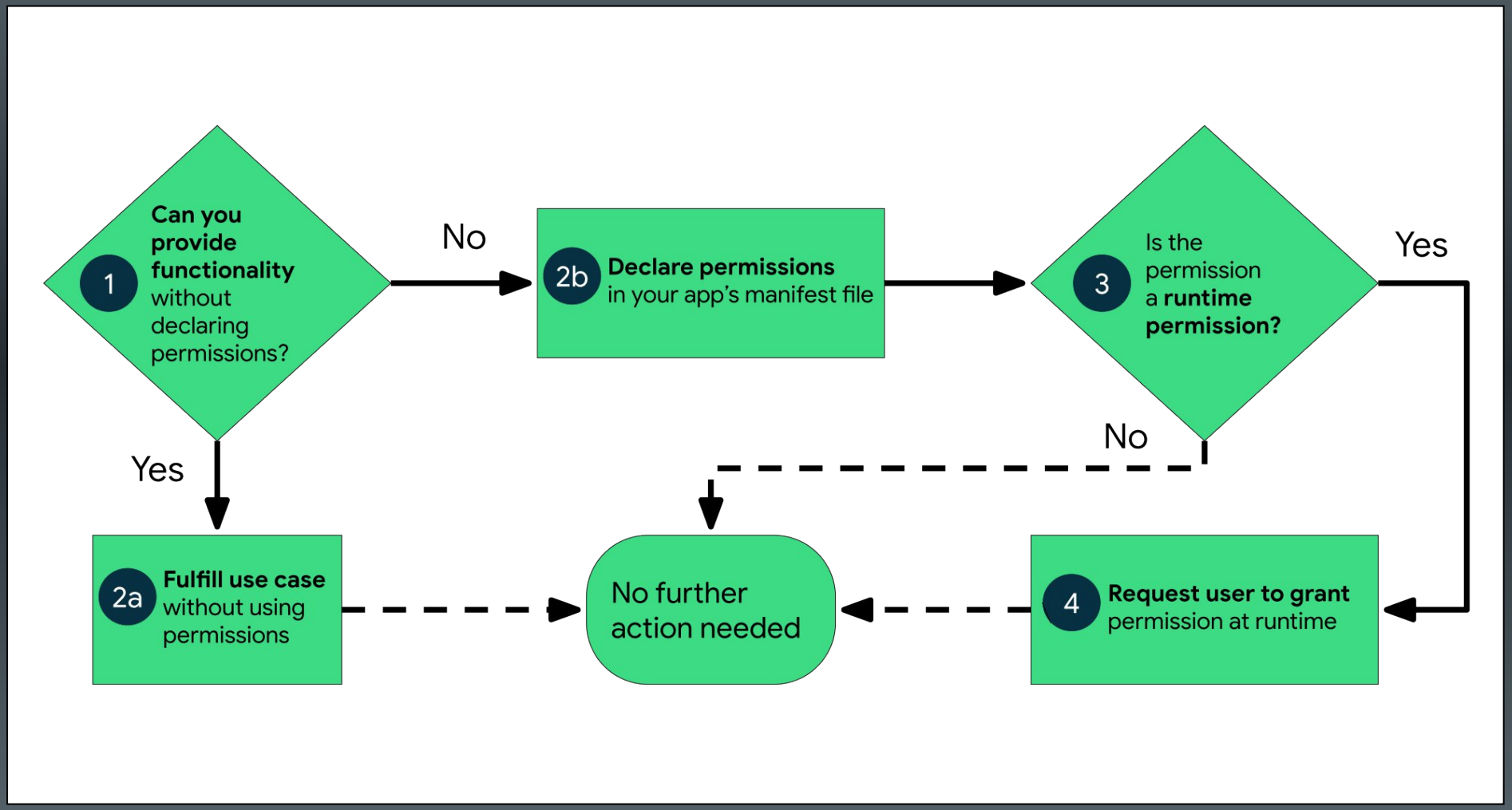
Permission "normale"

Permission "dangereuse"

Les permissions (4)

- Jusqu'à android 5.1 :
  - Les permissions sont validées à l'installation
  - Elles sont permanentes
- À partir d'android 6.0 (api 23) :
  - permissions d’installation : attribuées automatiquement à l’installation de l’application
  - permissions d’exécution : les permissions sont validées pendant l'exécution, au moment où elles sont requises la première fois
  - Possibilité de retirer des permissions données à une application
    - **Implique de vérifier la disponibilité des permissions dans le code**

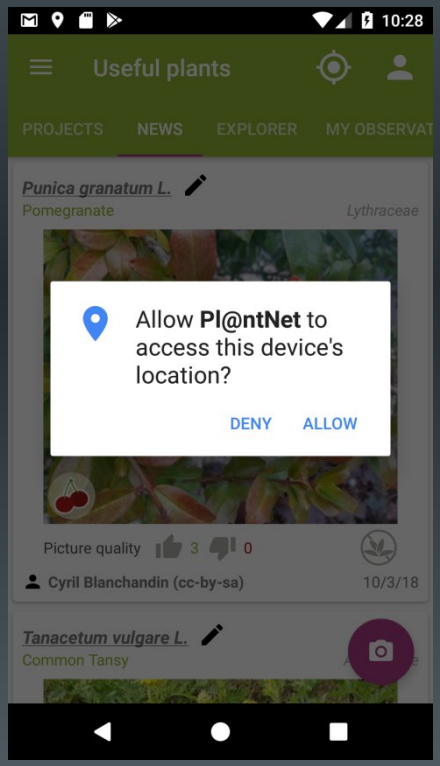
Les permissions (5) - Workflow

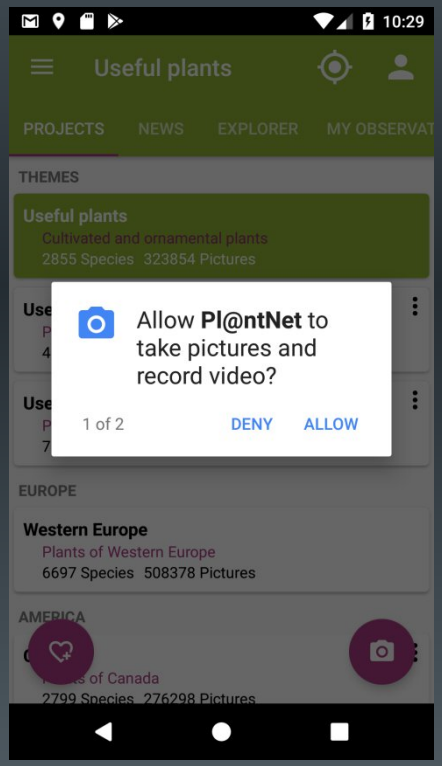


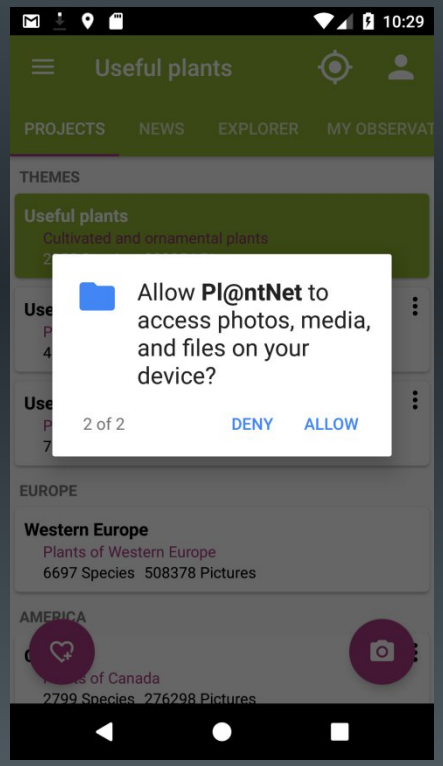
source : https://developer.android.com/guide/topics/permissions/overview?hl=fr

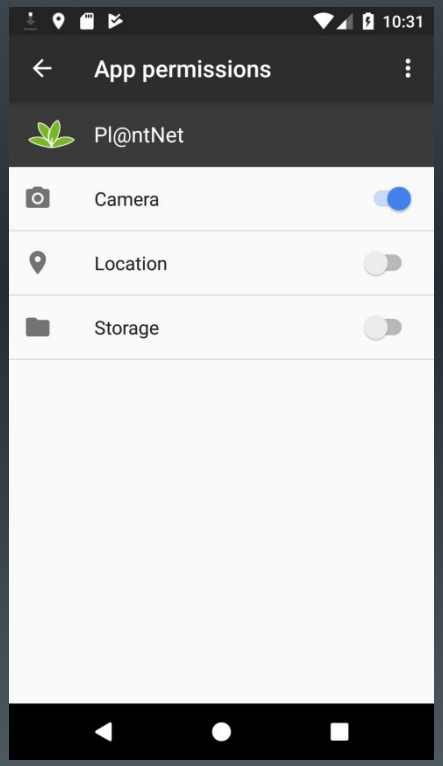
Les permissions (6) - Exemple

- Installation de PlantNet
  - Aucune demande de permission









Lancement de l'application : demande d'accès à la position

Appui sur le bouton « photo » : demande d'accès à la caméra et au stockage

Retrait de certains droits

Quelques permissions

- Définies dans Manifest.permission :
  - ACCESS_NETWORK_STATE, ACCESS_WIFI_STATE
  - SET_ALARM, SET_WALLPAPER
  - VIBRATE
  - ACCESS_COARSE_LOCATION, ACCESS_FINE_LOCATION
  - CAMERA
  - READ_EXTERNAL_STORAGE

normales

dangereuses

Présence d’une permission

```
ContextCompat.checkSelfPermission(Context, String)
```

Permission à tester
(cf classe Manifest.permission)

– Retourne :

```
PackageManager.PERMISSION_GRANTED

PackageManager.PERMISSION_DENIED
```

```
if (ContextCompat.checkSelfPermission(this, Manifest.permission.CAMERA)
     != PackageManager.PERMISSION_GRANTED) {

    // gérer l'accès non autorisé à la caméra

}
```

Demande d’une permission (1)

- Méthode 1 : Passer par la méthode :
  - `ActivityCompat.requestPermissions()`

L'activité demandeuse
(this)

Liste des permissions
(définies dans
Manifest.permission)

```
public static void requestPermissions( Activity activity,
                                       String[] permissions,
                                       int requestCode
                                     )
```

Identificateur de la demande
(int >= 0)

Demande d’une permission (2)

- Exemple 1 :

Identifiant interne de la demande de permissions

Liste des permissions à demander

```
public final int MY_PERMISSIONS_REQUEST = 1;

…

String [] liste = { Manifest.permission.CAMERA,
                    Manifest.permission.READ_EXTERNAL_STORAGE };

…
ActivityCompat.requestPermissions(this, liste, MY_PERMISSIONS_REQUEST);
```

Demande d'une permission (3)

- Exemple 2 :

Identifiant interne de la demande de permissions

```
public final int MY_PERMISSIONS_REQUEST_LOCATION = 2;

…

if (ContextCompat.checkSelfPermission(this,
                                      Manifest.permission.ACCESS_FINE_LOCATION)
        != PackageManager.PERMISSION_GRANTED) {

        ActivityCompat.requestPermissions(
                this,
                new String[]{Manifest.permission.ACCESS_FINE_LOCATION},
                             MY_PERMISSIONS_REQUEST_LOCATION);

} else {
    // permission déjà acquise …
}
```

Liste des permissions à demander
(*ici une seule permission*)

Demande d'une permission (4)

- Gérer la réponse
  - Surcharge d’une fonction callback

Identifiant de la demande de permission qui a été réalisée

Liste des permissions demandées

```
public void onRequestPermissionsResult(int requestCode,
                                       String permissions[],
                                       int[] grantResults)
```

Résultat pour chaque permission demandée (PERMISSION_GRANTED / PERMISSION_DENIED)

Demande d'une permission (5)

- Gérer la réponse
  - Exemple

```
@Override
public void onRequestPermissionsResult(int requestCode,
                                       String permissions[],
                                       int[] grantResults) {

    switch (requestCode) {
        case MY_PERMISSIONS_REQUEST_LOCATION : // gérer exemple 2
            // requête « cancelled » => tableau de résultat vide
            if (grantResults.length > 0
                && grantResults[0] == PackageManager.PERMISSION_GRANTED) {

                    // lancer l'utilisation du gps

            } else {
                    // gérer l'impossibilité d'utiliser le gps
            }
            break;
        case MY_PERMISSIONS_REQUEST : // gérer exemple 1
        …
    }// switch
}
```

Demande d'une permission (5)

- Méthode 2 : Passer par une classe de contrat
  - Pas d'identifiant de requête
  - Même principe que pour les intentions
    - Classe ActivityResultContracts.RequestPermission

La permission à demander
(Manifest.permission.XXX)

```
private ActivityResultLauncher<String>
  simplePermissionLauncher =  registerForActivityResult(
      new ActivityResultContracts.RequestPermission(),
      new ActivityResultCallback<Boolean>(){
         public void onActivityResult(Boolean res){
            ...
         }
      }
);
```

Le résultat de la demande :
- True : autorisation validée
- False : autorisation refusée

Demande d'une permission (6)

- Lancer la demande de permission

```
private ActivityResultLauncher<String>
  simplePermissionLauncher =  registerForActivityResult(
      new ActivityResultContracts.RequestPermission(),
      new ActivityResultCallback<Boolean>(){
         public void onActivityResult(Boolean res){
            ...
         }
      }
);
```

Lancement de la demande - récupérée par

```
if(ContextCompat.checkSelfPermission( this, Manifest.permission.CAMERA) ==
      PackageManager.PERMISSION_GRANTED) {
…
} else {
  // demander la permission
  simplePermissionLauncher.launch(Manifest.permission.CAMERA);
}
```

Demande d'une permission (7)

- Demande de permissions **multiples**
  - Classe ActivityResultContracts.RequestMultiplePermissions
  - Entrée :
    - String[ ] : liste des permissions demandées
  - Retour
    - Map<String,Boolean> : résultat pour chaque permission
      - True : la permission est accordée
      - False : la permission est refusée

Demande d'une permission (8)

```
private ActivityResultLauncher<String[]> multiplePermissionLauncher =
   registerForActivityResult(
         new ActivityResultContracts.RequestMultiplePermissions(),
         new ActivityResultCallback<Map<String,Boolean>>(){
               public void onActivityResult(Map<String, Boolean> res){
                     ...
               }
         });
```

```
…
String [] listePermission = {
   Manifest.permission.ACCESS_FINE_LOCATION,
   Manifest.permission.CAMERA };

multiplePermissionLaucher.launch(listePermission);
```

Plan du cours

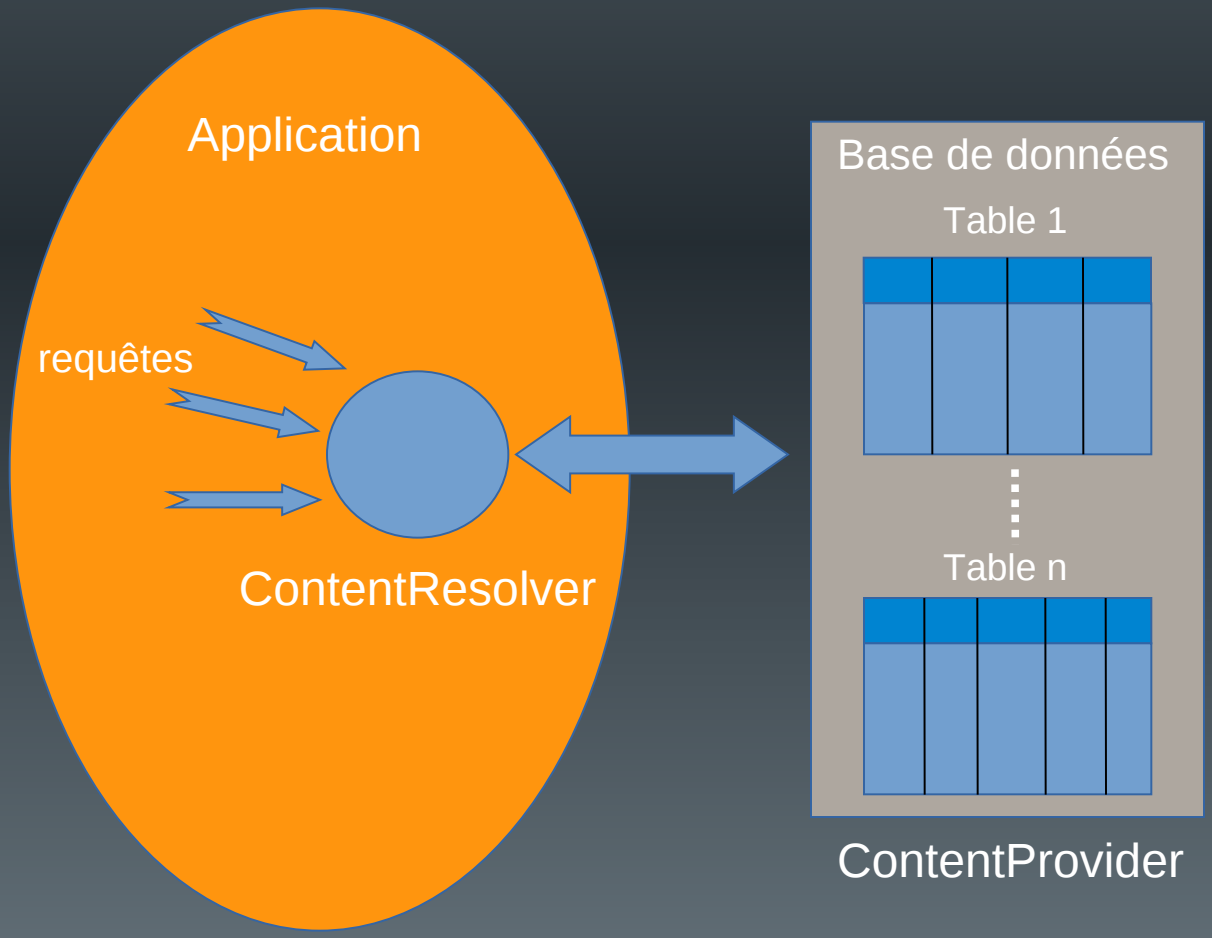
- Introduction
- Architecture d'une application Android
- Les activités
- Définir une interface graphique
- Les intentions explicites
- Les intentions implicites
- Les menus
- Les listes
- Les permissions
- Les content providers

Introduction

- Gestion des données par une application
  - Données locales/privées à une application
    - Sauvegardes dans des fichiers classiques
    - Sauvegardes dans des bases de données
      - SQL lite sous Android
  - Données extérieures à l'application (ex. contacts)
    - Accès via des fournisseurs de contenu
    - Nécessite de disposer des droits adéquats
  - Autoriser l'accès extérieur à des données de l'application
    - Créer un fournisseur de contenu

Les fournisseurs de contenu (1)

- Classe ContentProvider
  - Interface entre une application et des données



Les fournisseurs de contenu (2)

- Ils disposent
  - d'une URI permettant de les identifier : content://authority/path
    - authority : le nom du fournisseur (UserDictionary)
    - path : le nom des structures/sous-structures gérées
  - de données internes
    - Présentées à l'extérieur sous forme de tables

content://user_dictionary/words

Nom de la table

Nom du fournisseur de contenu

word	app id	frequency	locale	_ID
mapreduce	user1	100	en_US	1
precompiler	user14	200	fr_FR	2
applet	user2	225	fr_CA	3
const	user1	255	pt_BR	4
int	user5	100	en_UK	5

Les fournisseurs de contenu (3)

- Ils disposent
  - D'une classe de contrat, qui définit les constantes facilitant la manipulation du fournisseur
    - URI
    - Nom des colonnes des tables
    - Etc.
  - Exemple : classe UserDictionary.Words
    - UserDictionary.Words.CONTENT_URI (content://user_dictionary/words)
    - UserDictionary.Words.WORD
    - UserDictionary.Words.FREQUENCY

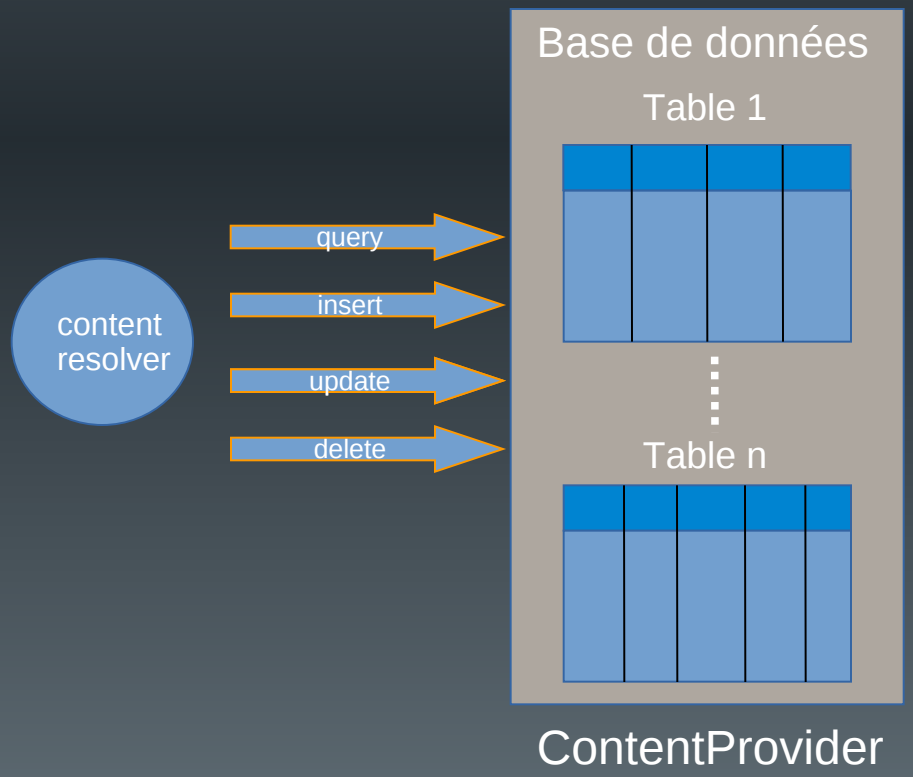
word	app id	frequency	locale	_ID
mapreduce	user1	100	en_US	1
precompiler	user14	200	fr_FR	2
applet	user2	225	fr_CA	3
const	user1	255	pt_BR	4
int	user5	100	en_UK	5

Les fournisseurs de contenu (4)

- Ils disposent
  - De permissions
    - Définies par le concepteur du fournisseur
    - Rendues publiques dans la « documentation »
    - Exemple :
      - android.permission.READ_USER_DICTIONARY
      - android.permission.WRITE_USER_DICTIONARY
  - Doivent être demandées dans le manifeste de l'application

Les content resolvers (1)

- Seul mécanisme d'accès aux content providers
- Disposent des méthodes d'accès aux fournisseurs
  - query()
  - insert()
  - update()
  - delete()



Les content resolvers (2)

- Création :

```
ContentResolver resolver = getContentResolver();
```

Objet pouvant interagir avec n'importe quel fournisseur de contenu

→ On fournit l'URI du fournisseur quand on souhaite l'utiliser ...

Exemples :

```
resolver.delete(UserDictionary.Words.CONTENT_URI, ...) ;
resolver.insert(Calendars.CONTENT_URI, ...) ;
resolver.update(Contacts.Phones.CONTENT_URI, ...) ;
```

Les content resolvers (3)

- La requête query()

L'URI du fournisseur de contenu.

La liste des noms de colonnes du fournisseur à récupérer.
(null = toutes les colonnes)

```
final Cursor  query( Uri uri,
                     String[] projection,
                     String   selection,
                     String[] selectionArgs,
                     String   sortOrder)
```

Filtre de sélection : clause WHERE de SQL sans le mot WHERE
(null = toutes les lignes)

Ordre de tri : clause ORDER BY de SQL sans les mots ORDER BY
(null = ordre par défaut)

Arguments de sélection (les « ? » qui apparaissent dans la sélection sont remplacés par ces arguments)

Les content resolvers (4)

- Le paramètre **`selection`**
  - Une chaîne contenant l'intégralité de la clause WHERE de la requête SQL
    - Inclus les critères liés (AND, OR, LIKE, …)
    - Pas de mot-clé WHERE
  - Problèmes de sécurité

```
String mySelection = " var = "  + userInput;
avec userInput valant " nothing; DROP TABLE *;"
```

    - Injection de code SQL
    - Utilisation du paramètre **`selectionArgs`**
      - Contient les valeurs à sélectionner
        - Interprétées comme des String, pas comme des requêtes SQL

```
String mySelection = " var1 = ? AND var2 = ?" ;
```

```
String mySelectionArgs[] = {"toto" ,"titi" };
```

Les content resolvers (5)

- La requête query() : exemples

```
String[] projection = {
  Calendars.CALENDAR_LOCATION,
  Calendars.CALENDAR_TIME_ZONE
}

String selection =
  Calendars.CALENDAR_LOCATION + " = ?" ;

String[] selectionArgs = { " France" }

resolver.query(Calendars.CONTENT_URI,
                projection,
                selection,
                selectionArgs,
                Calendars.CALENDAR_LOCATION+"
ASC" ) ;
```

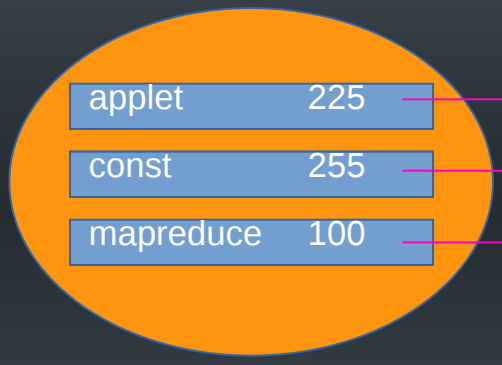
```
resolver.query(Calendars.CONTENT_URI,
                projection,
                null, null,null) ;
```

Retourne un objet de type Cursor

SELECT CALENDAR_LOCATION, CALENDAR_TIMEZONE FROM Calendars
WHERE CALENDAR_LOCATION='France' ORDER BY CALENDAR_LOCATION ASC

Les curseurs

- Objet représentant le résultat d'une requête
  - Liste des enregistrements (et colonnes) sélectionnés



word	app id	frequency	locale	_ID
mapreduce	user1	100	en_US	1
precompiler	user14	200	fr_FR	2
applet	user2	225	fr_CA	3
const	user1	255	pt_BR	4
int	user5	100	en_UK	5

- Diverses classes utilisables
  - Cursor
  - SimpleCursorAdapter

La classe Cursor

- Classe de base pour la récupération

```
ContentResolver resolver = getContentResolver();
Cursor mcursor = resolver.query(…) ;
```

- Nombreuses méthodes
  - getCount()
  - getColumnIndex(String nomColonne)
  - getType(int index)
  - moveToFirst(), moveToLast, moveToNext(), ...

Index pour les données du curseur
(pas du ContentProvider)

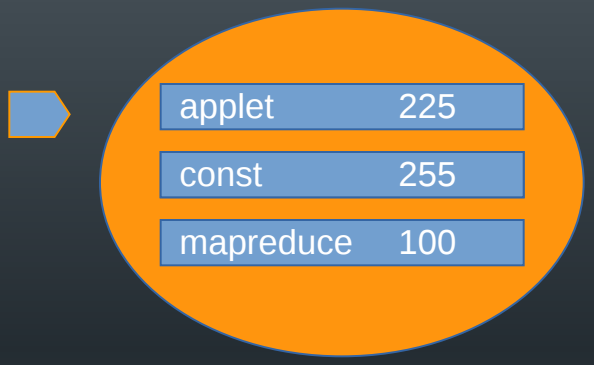
Gestion de la position du curseur dans la liste

La classe Cursor

- Exemple

UserDictionary.Words

word	app id	frequency	locale	_ID
mapreduce	user1	100	en_US	1
precompiler	user14	200	fr_FR	2
applet	user2	225	fr_CA	3
const	user1	255	pt_BR	4
int	user5	100	en_UK	5



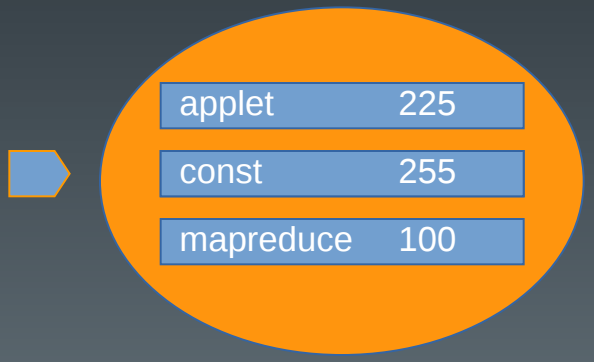
getCount() 3

getColumnIndex(UserDictionary.Words.FREQUENCY) 1

getString(0) applet

getInt(1) 225

moveToNext()



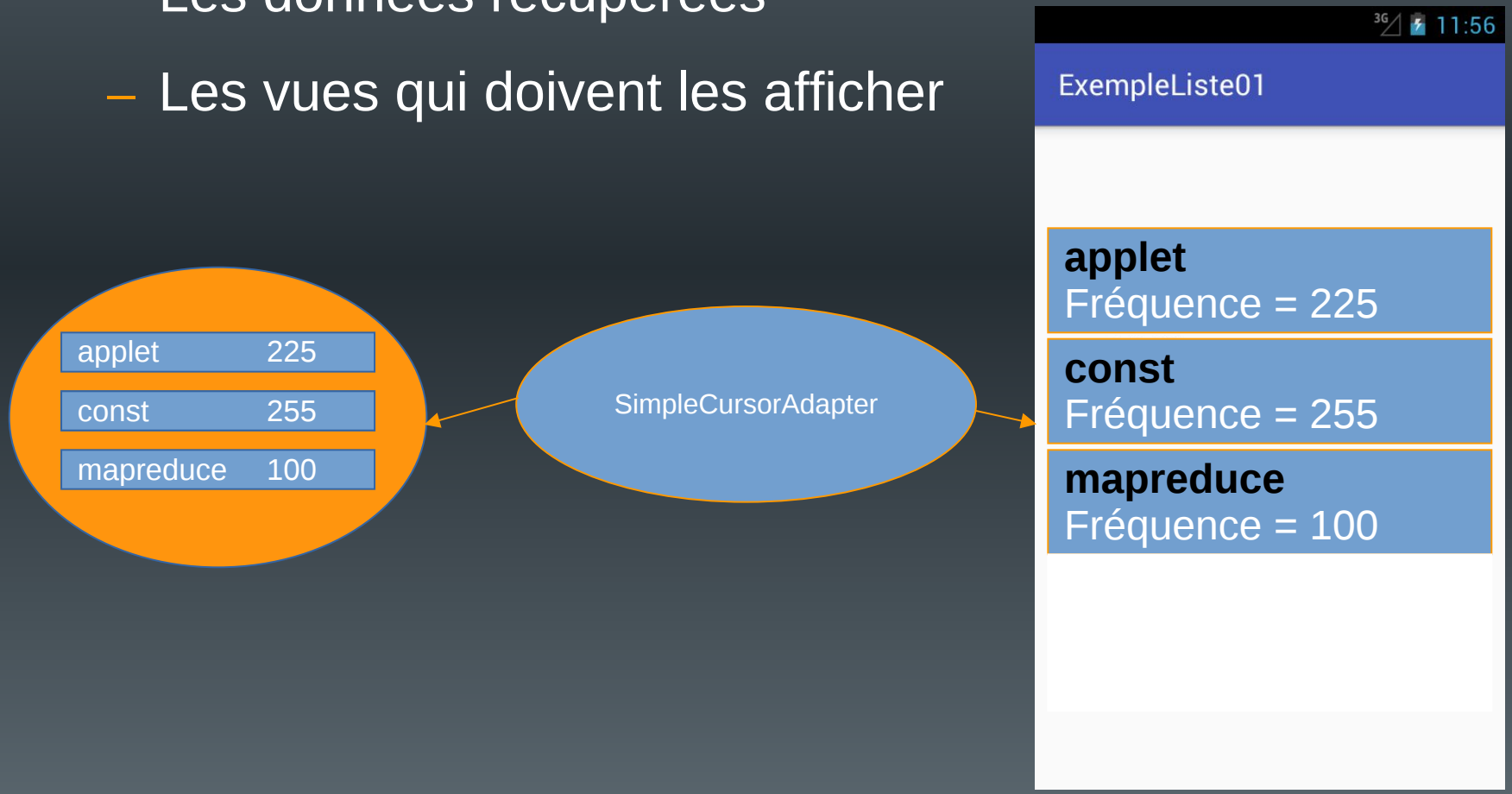
getString(0) const

getInt(1) 255

La position du curseur n'est pas initialisée tant qu'un déplacement n'a pas été effectué

La classe SimpleCursorAdapter

- Facilite le mapping entre :
  - Les données récupérées
  - Les vues qui doivent les afficher



La classe SimpleCursorAdapter

- Constructeur

```
SimpleCursorAdapter (Context context,
                     int layout,
                     Cursor c,
                     String[] from,
                     int[] to,
                     int flags)
```

La mise en page de chaque ligne de données extraite

Les données extraites

Le nom des colonnes du fournisseur de contenu à utiliser

Le nom des vues du *layout* vers lesquelles envoyer les données

Flag gérant le comportement de l'adaptateur

La classe SimpleCursorAdapter

- Exemple

android.R.layout.two_line_list_item

android .R.id.text1
android.R.id.text2

```
Cursor monCurseur = resolver.query(…) ;

SimpleCursorAdapter  sca =
   new SimpleCursorAdapter(this,
                                    android.R.layout.two_line_list_item,
                                    monCurseur,
                                    new String[]  {
                                        Userdictionary.Words.WORD,
                                        Userdictionary.Words.FREQUENCY
                                    },
                                    new int[] {
                                         android.R.id.text1,
                                         android.R.id.text2
                                    }, 0) ;
```

TP 07

Source : ark.media.mit.edu/~mike/Vanguard2012/i/androidall.jpg